DESARROOLO DE INTERFACES

Primera aplicación completa en Flutter



Miriam Mendieta Racedo.-
2DAMS
11/04/2026

Informe de Documentación Técnica Integral

- Proyecto: Python_Quizz_Flash
- Versión: 0.0.1
- Fecha: 14 de Abril de 2026
- Autor: Miriam Mendieta Racedo.-
- IES Serra Perenxisa

ÍNDICE

INTRODUCCIÓN **1**

DISEÑO DE INTERFAZ **2**

ANÁLISIS Y DISEÑO UI/UX **2**

Visión del Proyecto 2
Objetivo 2
Público objetivo 2
Análisis de Competidores 2
Principios de Usabilidad 3
Arquitectura Emocional y Modelos Mentales 4
Arquitectura Emocional 4
Modelos Mentales 4
Jerarquía Visual 4
UI vs UX 4
Design Thinking 4
Guía de Estilo Visual 5

DESARROLLO DE LA SOLUCIÓN Y LÓGICA DE NEGOCIO **8**

Análisis del Sistema 8
Requisitos Funcionales(RF) 8
Requisitos No Funcionales(RNF) 13
Diagrama de Casos de Uso (UML) 18
Diagrama UML de Clases 21
Diagrama de Paquetes 30
Diagrama de Componentes 31
Diagrama de Secuencia 32
Diagrama de Estados 38
Diagrama de Despliegue 40
Arquitectura de Software , Patrones de Diseño y Modelo de Datos 43
Estructura del Proyecto 45
Gestión de Datos e Integración de Inteligencia Artificial 51
Stack Tecnológico 58
Seguridad del Sistema 60
Calidad, Despliegue y Conclusiones 64
Conclusión 76

BIBLIOGRAFÍA **79**

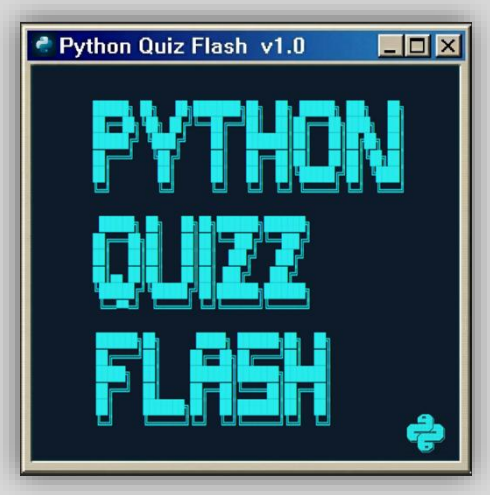
INTRODUCCIÓN

En la actualidad, las aplicaciones deben trascender la simple funcionalidad para ofrecer experiencias personalizadas, dinámicas e inmersivas. Este es el propósito del proyecto Python_Quizz_Flash, una aplicación educativa gamificada diseñada utilizando Flutter, el frame-work de código abierto de Google que permite crear aplicaciones con rendimiento nativo para móvil, web y escritorio desde una única base de código.

A diferencia de las herramientas de aprendizaje convencionales, Python_Quizz_Flash adopta una identidad visual inspirada en los entornos de desarrollo de los años 90 y en las terminales MS-DOS. Esta elección estética se traduce en una interfaz que simula la apariencia de un monitor CRT con scanlines, mientras que la arquitectura basada en widgets de Flutter asegura una usabi-lidad moderna e intuitiva.

Para garantizar la robustez y escalabilidad de la aplicación, se han integrado dos componentes tecnológicos clave. Primero, la conectividad mediante API permite que la aplicación se comuni-que con servicios externos para la gestión de datos en tiempo real, facilitando la persistencia del progreso del usuario y la operación de una economía virtual de "monedas". En segundo lugar, se ha introducido una capa de Inteligencia Artificial que utiliza Procesamiento de Lenguaje Natural (PLN) a través de una API, para generar contenido de explicación sobre conceptos de Python.

El desarrollo de Python_Quizz_Flash no solo se enfoca en satisfacer las necesidades del usuario, sino también en aplicar las mejores prácticas de desarrollo de software con el objetivo de crear una solución escalable y robusta que se adapte a futuras innovaciones tecnológicas, asegurando un rendimiento excepcional en cualquier dispositivo.



DISEÑO DE INTERFAZ

Análisis y Diseño UI/UX

Visión del Proyecto

Objetivo

El propósito central es desarrollar una herramienta cross-platform que permita practicar Python mediante un sistema de quiz gamificado, apoyado por Inteligencia Artificial para ofrecer tutoría personalizada y una estética cyberpunk que fomente la retención y motivación del usuario.

Público objetivo

Estudiantes de Python que necesitan repasar conceptos básicos de programación de una forma dinámica y amena. Esto incluye tanto a principiantes como a aquellos que buscan refrescar sus conocimientos.

La aplicación se segmenta en tres perfiles principales según su nivel técnico:

- ***Principiantes***: Necesitan perder el miedo al error mediante explicaciones detalladas.
- ***Intermedios***: Buscan desafiar sus conocimientos y profundizar en temas específicos con mini-cursos de IA.
- ***Avanzados:*** Aspiran al dominio total (nivel experto) y a la colección de logros complejos.

Análisis de Competidores

Se identificó que aplicaciones como ***Mimo*** o ***SoloLearn*** poseen una gamificación sólida pero carecen de una identidad visual inmersiva y de contenido generado por IA bajo demanda. El diferenciador clave de Python Quizz Flash es su estética retro terminal y su sistema de economía virtual (PY-Coins) vinculado a un tutor de IA.



Principios de Usabilidad

El diseño se basa en la filosofía "Retro en la forma, Moderno en el fondo", utilizando los siguientes principios de ***Heurísticas de Nielsen***:

1. ***Visibilidad del Estado del Sistema:***
   El usuario siempre conoce su progreso mediante barras de estado en el quiz, contadores de monedas visibles y estados de niveles (bloquea-do/completado).

2. ***Prevención de Errores:***
   - Incorporar confirmaciones antes de realizar acciones críticas, como el canje de monedas o la salida del juego.
   - Validar las respuestas del usuario en tiempo real, informando inmediatamente si hay algún error en la respuesta.

3. ***Consistencia:***
   Se aplica un sistema de diseño estricto donde los títulos siempre usan la fuente PressStart2P y el código FiraCode, manteniendo colores neón coheren-tes para-aciertos (verde) y errores (rojo).

4. ***Correspondencia con el mundo real:***
   Los entornos de desarrollo (IDE) suelen ser oscuros; la app replica esta con-vención para reducir la fatiga visual y generar familiaridad con el entorno profesional.

5. ***Flexibilidad y eficiencia:***
   Uso de navegación declarativa con GoRouter para acceso rápido y diseño res-ponsivo que se adapta a 6 plataformas distintas (móvil, web y escritorio).

Primera aplicación completa en Flutter.-

Arquitectura Emocional y Modelos Mentales

Arquitectura Emocional

La estética cyberpunk y los efectos de neón no son solo decorativos; buscan generar una sensación de "logro" y "curiosidad". El modo "Todo o Nada" añade adrenalina, convirtiendo el estudio en un desafío memorable.

Modelos Mentales

La app utiliza la metáfora de un videojuego retro de 8 bits para que el usuario asocie "aprender" con "jugar", reduciendo la resistencia psicológica al estudio de conceptos complejos.

Jerarquía Visual

Se establecen niveles de prioridad claros. La información crítica (títulos y alertas) destaca por su brillo y tipografía pixelada, mientras que el contenido de lectura usa fuentes monoespaciadas más legibles.

UI vs UX

Mientras la UI se enfoca en la entrega visual de la "Matrix" de Python, la UX garantiza que el flujo de aprendizaje sea fluido, sin barreras de entrada (offline-first) y con una curva de dificultad progresiva.

Design Thinking

- ***Problema***: Alta tasa de abandono en el aprendizaje de Python debido a retroalimentación genérica y falta de compromiso emocional.
- ***Fases:*** Se transitó desde la Empatía (entender el bloqueo de la página en blanco) hasta el Prototipado de una solución que ofrece pistas comprables y explicaciones pedagógicas (Concepto → Analogía → Código).
- ***Solución:*** Una app que utiliza IA para cerrar la brecha teoría-práctica, permitiendo al usuario solicitar contenido justo cuando lo necesita.

Primera aplicación completa en Flutter.-

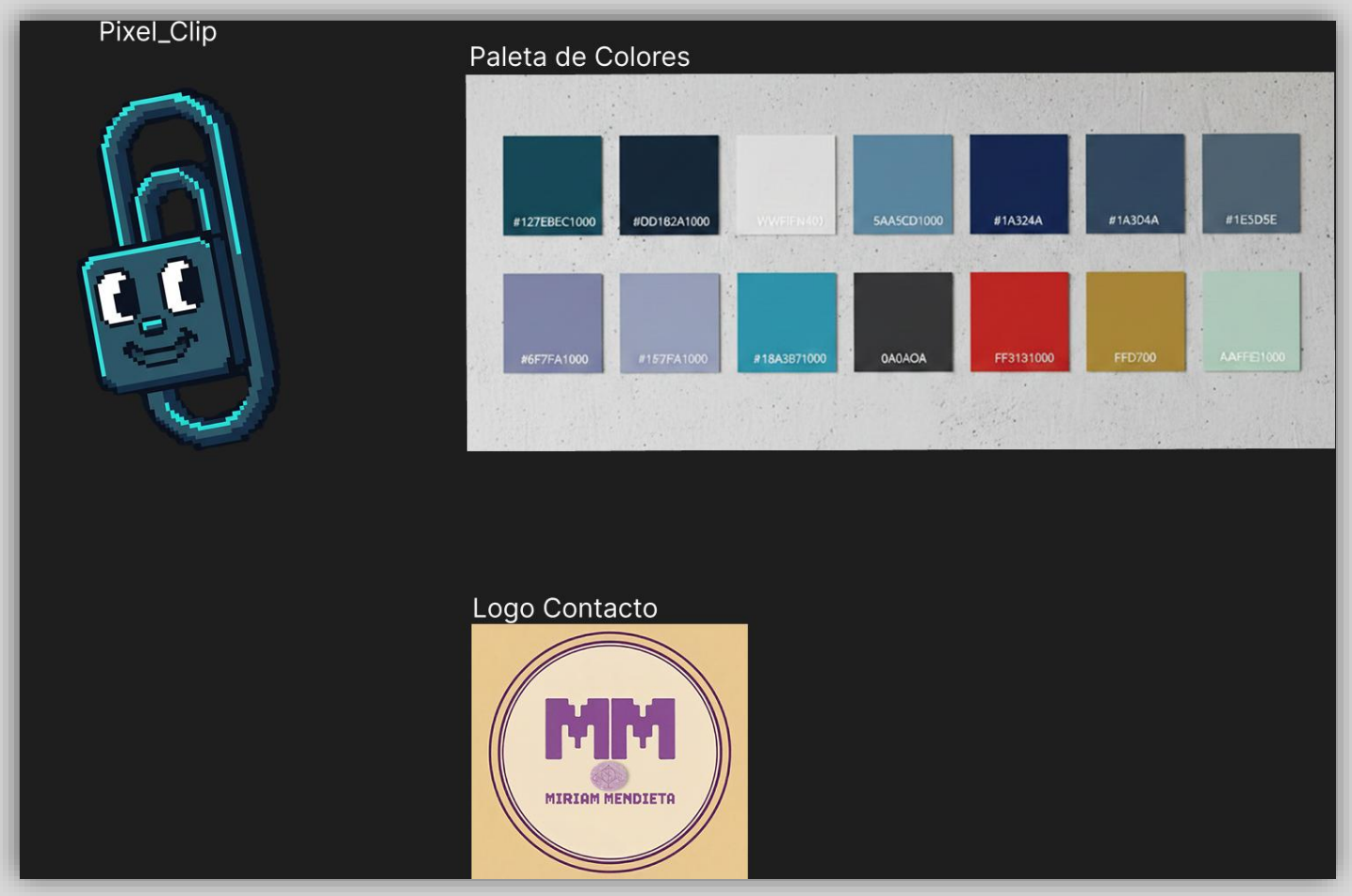
Guía de Estilo Visual

Tipografías

- Títulos: Press Start 2P (pixel art)
- Código: Fira Code (monospace)
- Cuerpo: IBM Plex Mono, Roboto Mono

Paleta de Colores (Dark Mode)

- Fondo Base: #000000 (Negro Puro) con efectos de scanlines CRT
- Texto Principal: #32FF00 (Verde Neón) / #00F5FF (Cian Brillante)
- Botones Principales: #32FF00 (Verde Neón)
- Advertencias/Errores: #FF3131 (Rojo Vibrante)
- Pixel Clip: #BD00FF (Púrpura Neón)
- Elementos Inactivos: #AAAAAA / #555555



Primera aplicación completa en Flutter.-

Prototipo y Wireframes (Figma)

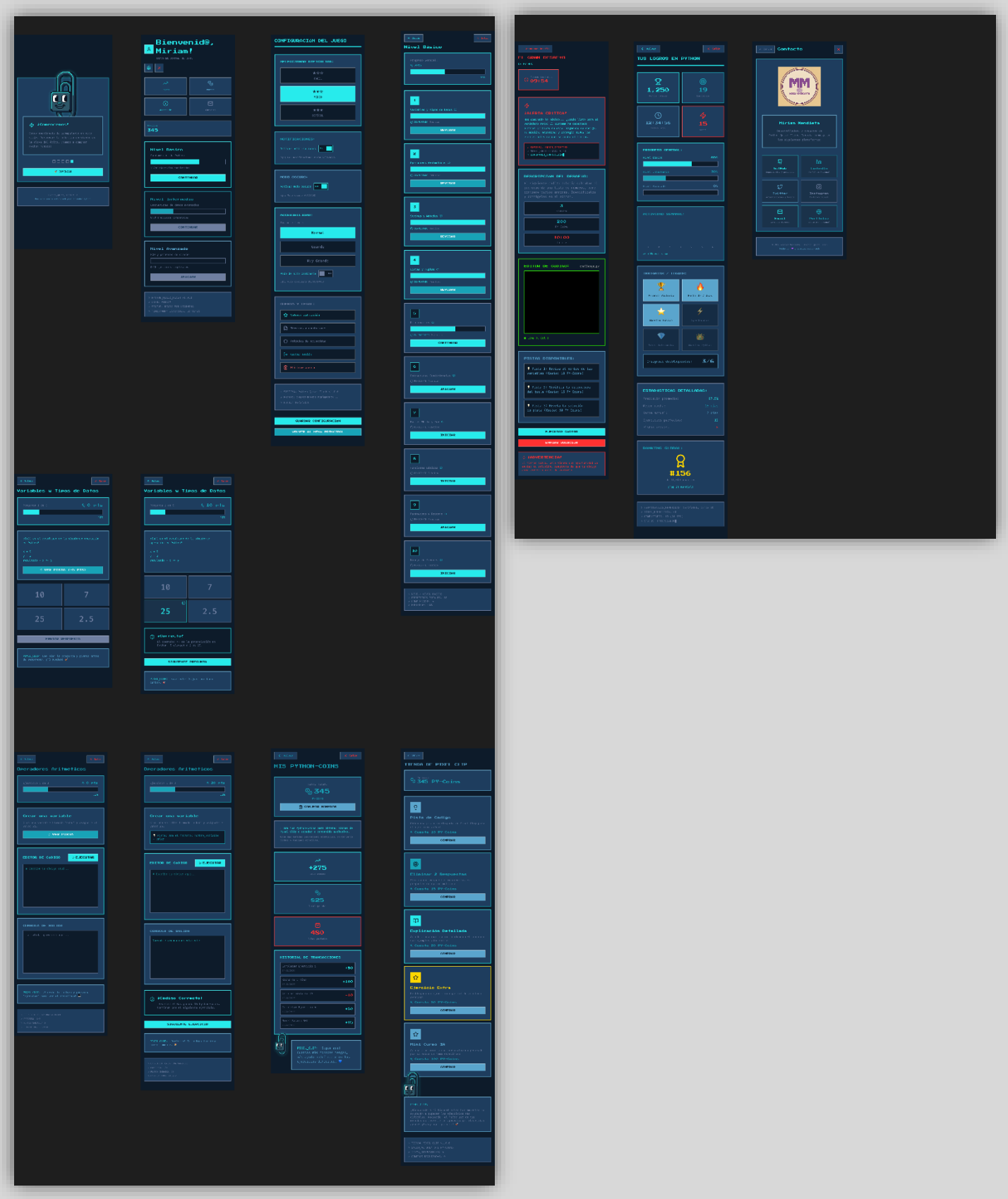
- Formulario de Registro: Implementado con Firebase Auth, incluyendo límites de intentos para seguridad.

- Mecanismo de la App: Sistema de 20 ejercicios aleatorios por sesión, con retroalimentación animada de 200-300ms.

- Wireframes y Prototipos: Evolución desde bocetos de baja fidelidad hasta un prototipo de alta fidelidad con 20+ capturas y flujos de usuario validados.



Primera aplicación completa en Flutter.-



Desarrollo de la Solución y Lógica de Negocio

Análisis del Sistema

Requisitos Funcionales(RF)

Los requisitos funcionales describen **qué debe hacer** el sistema. Son las capacidad des y características que el usuario final puede utilizar directamente.

RF-001: Sistema de Autenticación de Usuarios	
RF-001.1	El sistema debe permitir el registro de nuevos usuarios con nombre de usuario (nick), email y contraseña (mínimo 12 caracteres).
RF-001.2	El sistema debe permitir el inicio de sesión usando email O nombre de usuario indistintamente.
RF-001.3	El sistema debe verificar la disponibilidad del nick en tiempo real durante el registro.
RF-001.4	El sistema debe implementar rate limiting: bloquear temporalmente la cuenta durante 15 minutos tras 5 intentos fallidos consecutivos.
RF-001.5	El sistema debe permitir el cierre de sesión del usuario actual.
RF-001.6	El sistema debe permitir la eliminación permanente de la cuenta (Firebase Auth + Firestore), requiriendo re-autenticación reciente por seguridad.
RF-001.7	El sistema debe persistir la sesión del usuario tras recargar la página (especialmente en web mediante “Persistence.LOCAL”).

RF-002: Sistema de Quiz Interactivo	
RF-002.1	El sistema debe presentar ejercicios de 4 tipos diferentes: * Opción múltiple (multipleChoice): Selección de una respuesta correcta entre varias opciones * Completar código (fillInTheBlank): Escribir la parte faltante de un fragmento de código. * Detección de bugs (bugFix): Identificar y corregir errores en código existente. * Escritura de código (coding): Escribir código desde cero para resolver un problema.
RF-002.2	El sistema debe ofrecer 4 niveles de dificultad: básico, intermedio, avanzado y experto.
RF-002.3	Cada nivel debe contener 20 ejercicios normales + 1 ejercicio de reto final (21 en total).
RF-002.4	Los ejercicios deben seleccionarse aleatoriamente desde un pool predefinido en JSON (“assets/data/exercises_pool.json”).
RF-002.5	El sistema debe validar las respuestas del usuario: * Para opción múltiple: comparación exacta con la respuesta correcta. * Para fill-in-the-blank: comparación exacta o verificación si la respuesta termina con la correcta. * Para coding: verificación de keywords esenciales de Python (“print”, “len”, “def”, “if”, “for”, “while”, “return”) + coincidencia de contenido.
RF-002.6	El sistema debe otorgar +10 PY-Coins por respuesta correcta normal y +200 PY-Coins por reto final completado.
RF-002.7	El sistema debe permitir el uso de pistas (costo: 5 monedas) que muestran la pista obligatoria de cada ejercicio.
RF-002.8	El sistema debe permitir el uso del comodín "quitar 2 opciones incorrectas" (costo: 20 monedas) exclusivamente en ejercicios de opción múltiple.
RF-002.9	El sistema debe guardar el progreso del usuario (índice de ejercicio actual) en Firestore para permitir reanudar el quiz posteriormente.
RF-002.10	El sistema debe completar exitosamente el nivel si el usuario comete menos de 5 errores; de lo contrario, debe borrar el progreso del nivel.

RF-003: Modo Desafío "Todo o Nada"	
RF-003.1	El sistema debe ofrecer un desafío cronometrado de 10 minutos al final de cada nivel (básico, intermedio, avanzado).
RF-003.2	El desafío debe consistir en un ejercicio de programación (coding) de alta complejidad.
RF-003.3	Si el usuario falla el desafío, el sistema debe reiniciar TODAS las monedas a cero (pérdida total).
RF-003.4	Si el usuario completa el desafío exitosamente, debe otorgar +200 monedas y desbloquear el siguiente nivel.

RF-004: Sistema de Progresión de Niveles	
RF-004.1	El sistema debe implementar una cadena de desbloqueo secuencial: Básico → (desafío) → Intermedio → (desafío) → Avanzado→ (desafío) → Experto.
RF-004.2	Al completar un nivel exitosamente, el sistema debe guardar un checkpoint de monedas y contadores en Firestore.
RF-004.3	Al fallar un nivel superior, el sistema debe restaurar las monedas y el progreso al checkpoint del nivel anterior (reset parcial).
RF-004.4	El nivel básico al fallar implica reset completo a cero (monedas, ejercicios, retos).

RF-005: Tienda de Contenido Generado por IA	
RF-005.1	El sistema debe permitir comprar 3 tipos de contenido generado por IA: * Explicaciones (costo: 20 monedas): Explicación detallada de un tema de Python con concepto, analogía, código y errores comunes. * Ejercicios adicionales (costo: 50 monedas): Ejercicio práctico con enunciado, 3 pistas, solución y casos de prueba. * Mini-cursos (costo: 100 monedas): Curso completo con objetivos, módulos, ejercicios y recursos.
RF-005.2	El sistema debe limitar la compra a 1 contenido por tipo por día (límite diario).
RF-005.3	El sistema debe permitir al usuario seleccionar el tema y nivel de dificultad del contenido a generar.
RF-005.4	El sistema debe visualizar el contenido generado en una pantalla dedicada con formato estructurado.
RF-005.5	El sistema debe guardar el historial de contenido generado en Firestore para consulta posterior.

RF-006: Sistema de Economía (PY-Coins)	
RF-006.1	El sistema debe mantener un balance de monedas virtual (PY-Coins) para cada usuario.
RF-006.2	El sistema debe registrar el total histórico de monedas ganadas y gastadas
RF-006.3	El sistema debe rastrear el gasto desglosado en pistas y compras de tienda.
RF-006.4	El sistema debe persistir los datos de economía localmente (SharedPreferences) para actualizaciones instantáneas.
RF-006.5	El sistema debe sincronizar los datos de economía con Firebase Firestore en segundo plano para consistencia multi-dispositivo.

RF-007: Sistema de Logros y Estadísticas	
RF-007.1	El sistema debe mostrar 7 badges/logros vinculados a desafíos completados.
RF-007.2	El sistema debe mostrar estadísticas detalladas: ejercicios completados, fallados, retos superados, monedas totales ganadas/gastadas.
RF-007.3	El sistema debe mostrar el historial de transacciones de monedas.

RF-008: Configuración de la Aplicación	
RF-008.1	El sistema debe permitir activar/desactivar la música de fondo.
RF-007.2	El sistema debe permitir activar/desactivar los efectos de sonido.
RF-007.3	Las preferencias deben persistirse usando almacenamiento seguro (“FlutterSecureStorage”).

RF-009: Navegación y Enrutamiento	
RF-009.1	El sistema debe implementar enrutamiento declarativo usando GoRouter con 17 rutas definidas.
RF-009.2	El sistema debe navegar entre pantallas usando query parameters (ej: “?difficulty=basic”) y datos extra (objetos “AiContent”).
RF-009.3	El sistema debe verificar el estado de autenticación en la pantalla de splash y redirigir automáticamente a la pantalla correspondiente.

RF-010: Pantalla de Inicio (Onboarding)	
RF-010.1	El sistema debe implementar enrutamiento declarativo usando GoRouter con 17 rutas definidas. El sistema debe mostrar una pantalla de introducción/tutorial para usuarios nuevos con diapositivas explicativas sobre cómo usar la app.

Requisitos No Funcionales(RNF)

Los requisitos no funcionales describen **cómo debe comportarse** el sistema en términos de calidad, rendimiento, seguridad y restricciones técnicas.

RNF-001: Multiplataforma	
RNF-001.1	La aplicación debe ser compatible con Android, iOS, Web, Linux, macOS y Windows mediante Flutter.
RNF-001.2	La interfaz debe ser 100% responsiva y adaptable a diferentes tamaños de pantalla (móvil, tablet, desktop).

RNF-002: Rendimiento	
RNF-002.1	La carga inicial de la aplicación (splash screen) no debe exceder los 3 segundos en condiciones normales de red.
RNF-002.2	La selección y carga de 21 ejercicios desde el pool JSON debe completarse en menos de 1 segundo.
RNF-002.3	Las actualizaciones de UI tras responder una pregunta deben ser instantáneas (menos de 100ms) usando estado local antes de sincronizar con Firebase.
RNF-002.4	La generación de contenido por IA debe completarse en un máximo de 30 segundos (considerando los 3 intentos de fallback).

RNF-003: Seguridad	
RNF-003.1	Las contraseñas deben tener una longitud mínima de 12 caracteres (requisito de Firebase Auth).
RNF-003.2	El sistema debe implementar rate limiting de 5 intentos fallidos con bloqueo de 15 minutos para prevenir ataques de fuerza bruta.
RNF-003.3	Las API Keys (Gemini, Groq, Firebase) NO deben commitearse al repositorio; deben cargarse desde ".env" (móvil/escritorio) o "window.flutter_config" (web) inyectado vía JavaScript.
RNF-003.4	El sistema debe sanitizar todas las entradas de texto antes de enviarlas a la IA para prevenir ataques de Prompt Injection, usando 40+ patrones regex en inglés y español.
RNF-003.5	**:** Las Firestore Rules deben restringir el acceso a datos: cada usuario solo puede leer/escribir su propio documento en "users", "transactions", "ai_content" y "ai_usage".
RNF-003.6	La eliminación de cuenta debe requerir re-autenticación reciente (requerimiento de seguridad de Firebase Auth).
RNF-003.7	Los logs de producción deben ocultar datos sensibles (API Keys, contraseñas, PII) mediante sanitización automática.

RNF-004: Disponibilidad y Resiliencia	
RNF-004.1	El servicio de IA debe implementar estrategia de fallback en 3 niveles: Gemini 1.5 Pro → Gemini 2.0 Flash → Groq Llama 3.
RNF-004.2	La aplicación debe funcionar parcialmente sin conexión a internet (ejercicios cargados desde assets locales), aunque la sincronización de progreso requiere red.
RNF-004.3	En caso de error de Firebase durante la inicialización, la aplicación debe continuar funcionando sin detenerse (try-catch silencioso en "main.dart").

RNF-005: Persistencia de Datos	
RNF-005.1	Los datos de economía deben persistirse localmente con “SharedPreferences” para acceso instantáneo.
RNF-005.2	Las preferencias de usuario (música, SFX, vibración, idioma) deben persistirse con “FlutterSecureStorage”.
RNF-005.3	El progreso del quiz, monedas, logros y checkpoints deben persistirse en Cloud Firestore para disponibilidad multi-dispositivo.
RNF-005.4	El contenido generado por IA debe persistirse en Firestore para mantener historial accesible.

RNF-006: Mantenibilidad y Calidad del Código	
RNF-006.1	El código debe seguir las guías de estilo oficiales de Dart y Flutter (flutter_lints).
RNF-006.2	Cada archivo debe incluir comentarios explicativos línea por línea para documentación.
RNF-006.3	Los modelos de datos deben usar “freezed” para inmutabilidad automática y “json_serializable” para serialización.
RNF-006.4	La gestión de estado debe usar Riverpod con generación automática de providers (“riverpod_generator”).
RNF-006.5	El enrutamiento debe centralizarse en un único archivo (“app_router.dart”) usando GoRouter.

RNF-007: Experiencia de Usuario (UX)	
RNF-007.1	La interfaz debe mantener una estética cyberpunk/retro-terminal consistente en toda la aplicación.
RNF-007.2	Las fuentes deben ser legibles a pesar de ser pixel art: PressStart2P para títulos, IBMPlexMono/FiraCode para cuerpo y código.
RNF-007.3	Los colores neón deben tener suficiente contraste sobre el fondo azul oscuro (#0D1B2A) para cumplir con estándares de accesibilidad WCAG AA.
RNF-007.4	La música de fondo debe reproducirse automáticamente al iniciar la app (con workaround para autoplay en navegadores web).
RNF-007.5	El sistema de notificaciones debe usar toasts estandarizados (“app_notifications.dart”) para feedback visual al usuario.

RNF-008: Escalabilidad	
RNF-008.1	El pool de ejercicios debe ser fácilmente extensible añadiendo más entradas JSON sin modificar el código.
RNF-008.2	La arquitectura en capas (presentación → estado → dominio → servicios → infraestructura) debe facilitar añadir nuevas funcionalidades sin afectar componentes existentes.
RNF-008.3	El uso de ChangeNotifier para “EconomiaProvider” y Riverpod para providers reactivos permite escalar independientemente según complejidad .

RNF-009: Límites de APIs Externas	
RNF-009.1	Gemini API tiene un límite gratuito de 60 requests/minuto.
RNF-009.2	Groq API tiene un límite gratuito de 30 requests/minuto.
RNF-009.3	El sistema debe controlar el uso diario de IA (1 contenido/tipo/día) para no exceder cuotas gratuitas.

RNF-010: Seguridad de la Capa de IA	
RNF-010.1	**Sanitización de Entrada:** El sistema debe filtrar el input del usuario mediante una lista negra de 40+ patrones regex que detecten comandos de cambio de rol (ej: "Ignore previous instructions", "Eres ahora un admin").
RNF-010.2	**Prevención de Fuga de Keys:** El sistema debe interceptar cualquier respuesta de la IA que contenga patrones similares a API Keys, Tokens o URIs de base de datos antes de mostrarla al usuario.
RNF-010.3	**Aislamiento de Contexto:** El System Prompt debe enviarse en una estructura de mensaje protegida (Role: System) separada del mensaje del usuario (Role: User), impidiendo que el usuario sobrescriba las reglas del sistema.
RNF-010.4	**Control de Estado Inmutable:** La IA no debe tener permisos de escritura directa sobre el estado de la aplicación (monedas, nivel). El sistema solo aceptará datos de la IA si cumplen un esquema JSON estricto validado por código.
RNF-010.5	**Detección de Inyección Indirecta**: Al generar ejercicios, el sistema debe verificar que el código Python propuesto por la IA no contenga librerías peligrosas (os, subprocess, shutil) que puedan ejecutarse en un entorno de prueba.

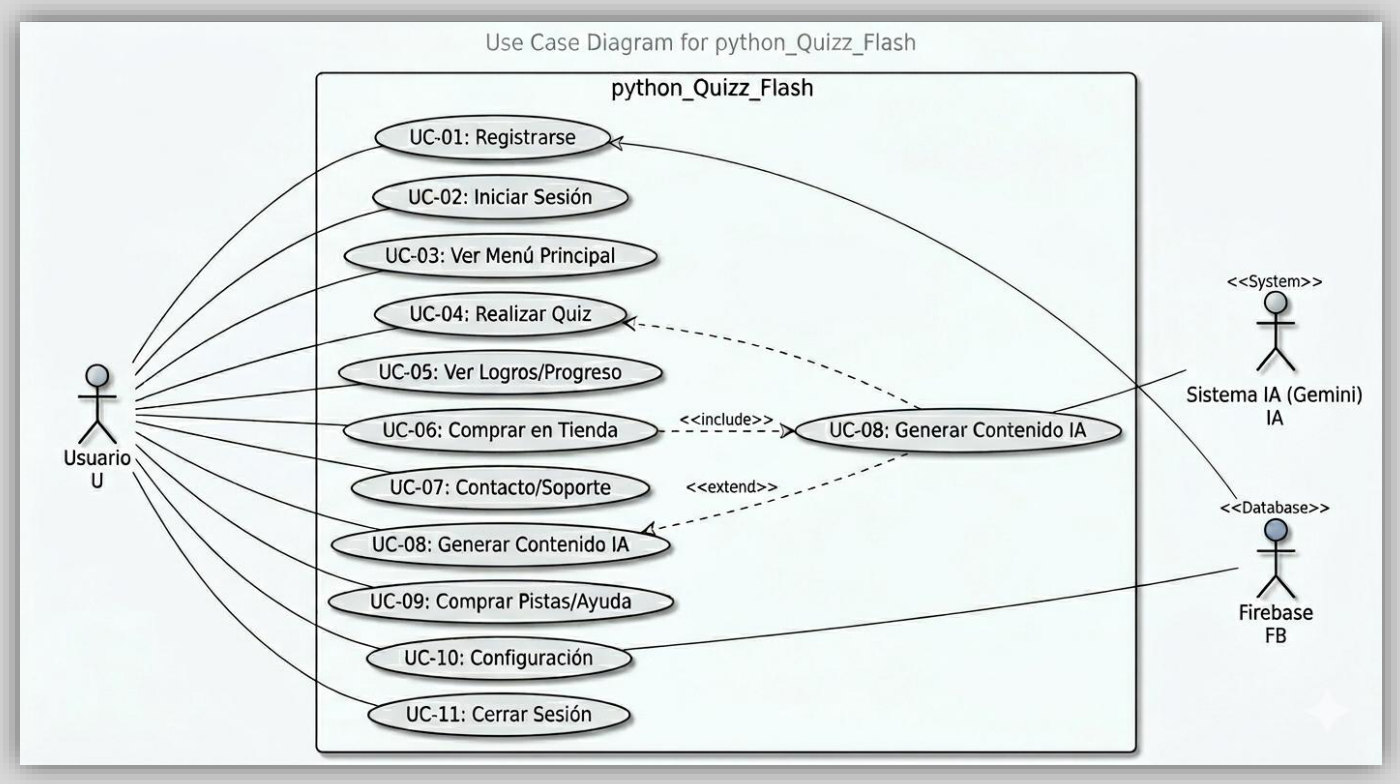
Primera aplicación completa en Flutter.-

Diagrama de Casos de Uso (UML)

Interacción entre el Usuario, el Sistema y la IA.

ACTORES

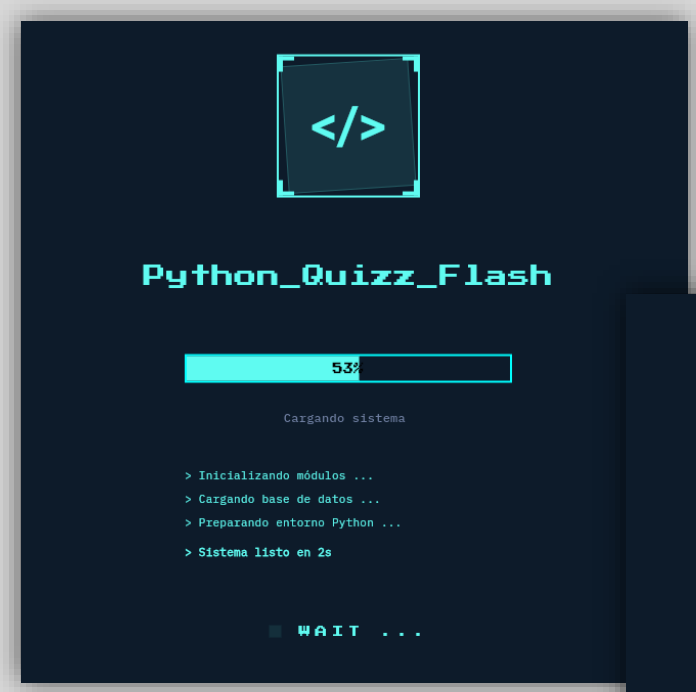
Usuario Sistema IA Firebase

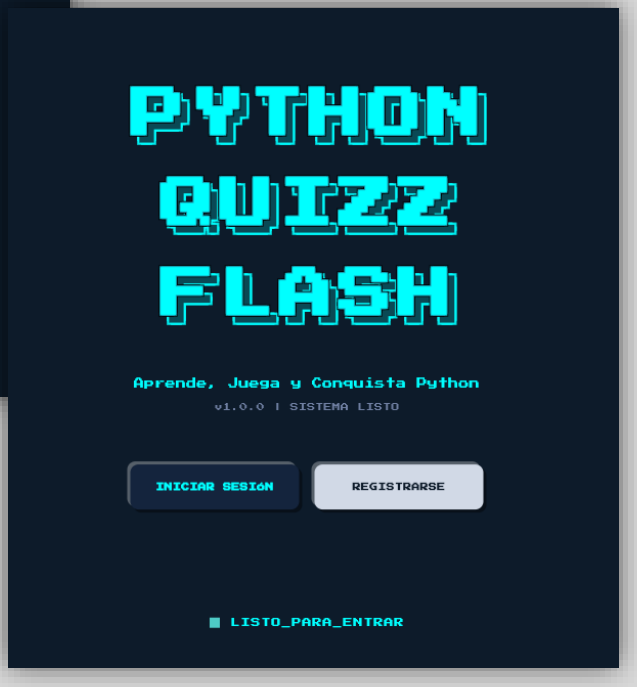


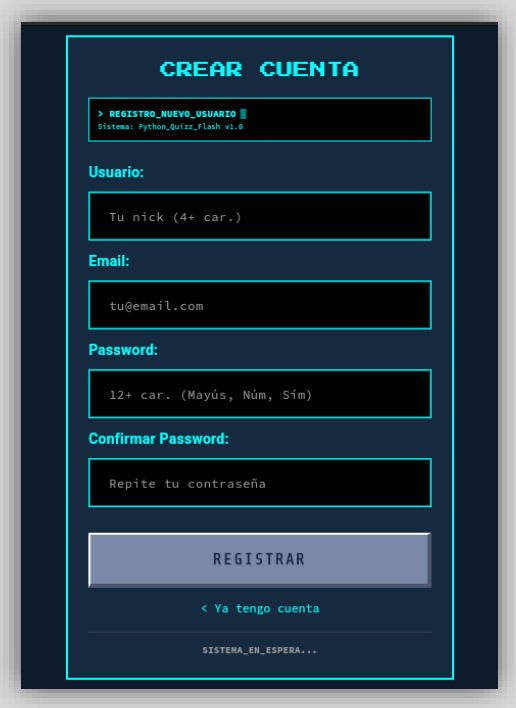
Descripción de Casos de Uso

Caso de Uso	***Descripción***	***Precondición***	***Postcondición***
Registrarse	Crea una cuenta nueva con nick, email y password	No tener cuenta previa	Usuario creado en Firebase Auth y Firestore
Iniciar Sesión	Accede a su cuenta con email o nick	Tener cuenta registrada	Sesión activa, redirige a Main Menu
Jugar Quiz	Completa ejercicios de Python en un nivel	Estar autenticado	Progreso guardado, monedas ganadas
Comprar Contenido IA	Adquiere explicaciones, ejercicios o mini-cursos	Tener monedas suficientes + no exceder límite diario	Contenido generado y guardado en Firestore
Usar Pistas	Compra ayuda durante el quiz	Tener monedas suficientes	Pista mostrada u opciones eliminadas, monedas restadas
Progresar de Nivel	Completa un nivel con < 5 errores	Completar los 21 ejercicios	Checkpoint guardado, siguiente nivel desbloqueado
Completar Desafío	Resuelve reto "todo o nada" cronometrado	Nivel anterior completado	+200 monedas o pérdida total + siguiente nivel
Ver Logros	Consulta badges, estadísticas e historial	Estar autenticado	Visualización de métricas personales
Configurar App	Ajusta música, SFX, vibración, idioma	Estar en la app	Preferencias guardadas en SecureStorage
Eliminar Cuenta	Borra permanentemente su cuenta	Re-autenticación reciente	Datos eliminados de Firestore + Firebase Auth

Primera aplicación completa en Flutter.-







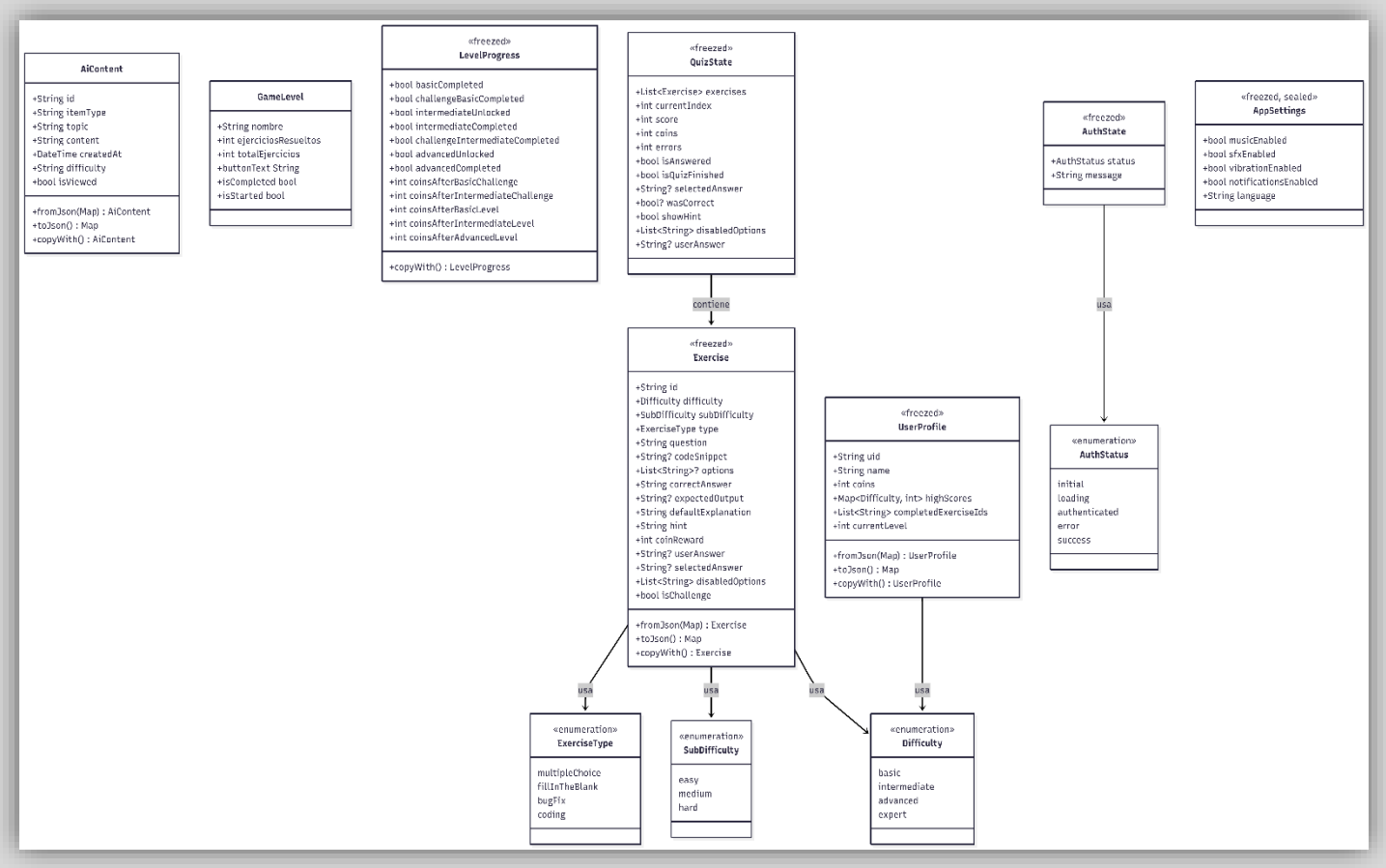


Primera aplicación completa en Flutter.-

Diagrama UML de Clases

Representa cómo se estructuran los datos y cómo fluye la lógica entre los componentes de Flutter.

1. **Modelos de Datos:** Clases puras (UserProfile, Exercise, GameLevel) que definen la estructura de la información y su serialización JSON.



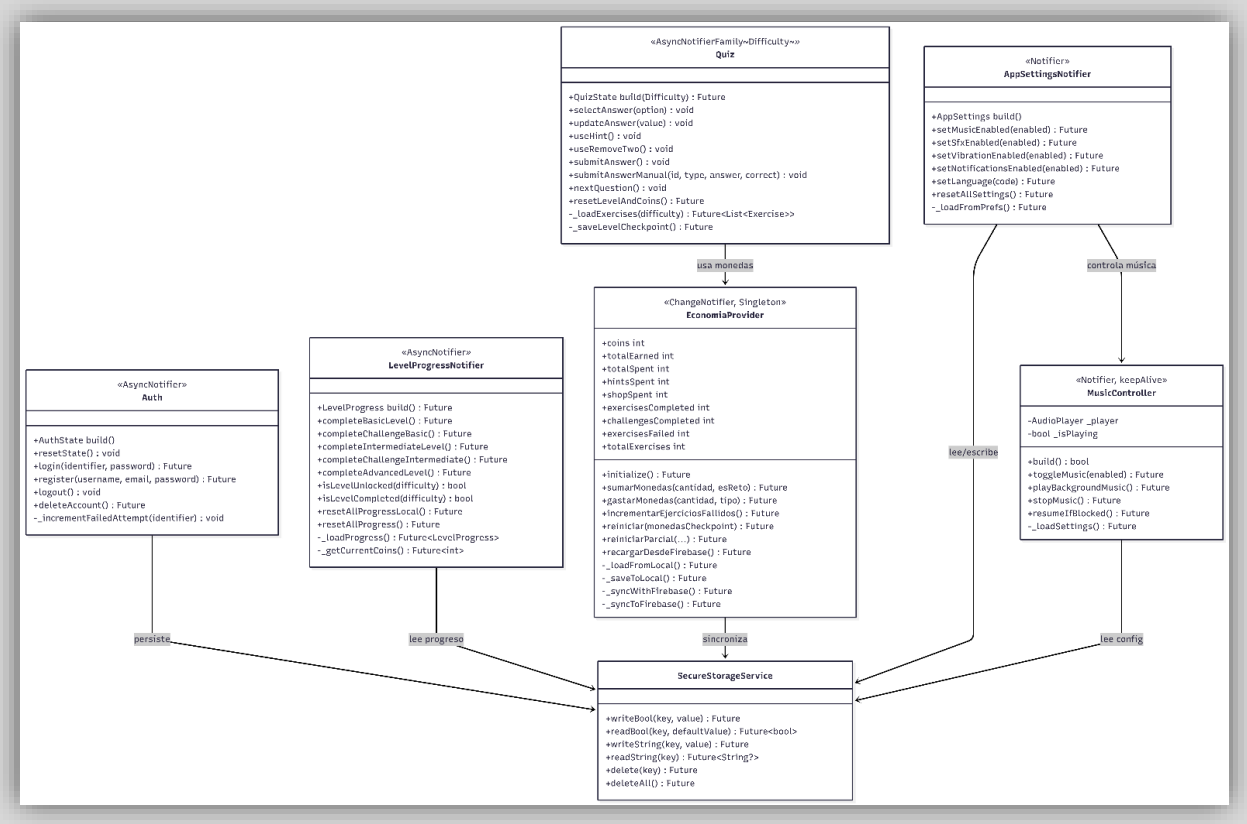
Este diagrama muestra los **modelos de datos principales** de la aplicación:

- **Exercise**: Modelo inmutable generado por Freezed que representa una pregunta del quiz. Contiene 16 campos que definen desde la dificultad hasta la respuesta del usuario. Usa convertidores JSON personalizados para los enums (“DifficultyConverter”, “ExerciseTypeConverter”).

- **Difficulty, SubDifficulty, ExerciseType**: Enums que clasifican los ejercicios por nivel de complejidad y tipo de interacción.

- **UserProfile**: Modelo del perfil del usuario con sus monedas, puntuaciones y ejercicios completados.
- **AiContent**: Representa contenido generado por IA (explicaciones, ejercicios, mini-cursos). Es una clase manual (no Freezed) con patrón de co pia inmutable.
- **GameLevel**: Modelo simple que representa el progreso de un nivel con getters computados para determinar el estado.
- **LevelProgress**: Estado detallado del progreso por nivel, usado por el "Le velProgressNotifier".
- **QuizState**: Estado completo del quiz actual con ejercicios, puntuación, errores y opciones deshabilitadas.
- **AuthState**: Estado simplificado de autenticación con solo status y mensaje.
- **AppSettings**: Clase sellada (sealed) con configuraciones de la app.

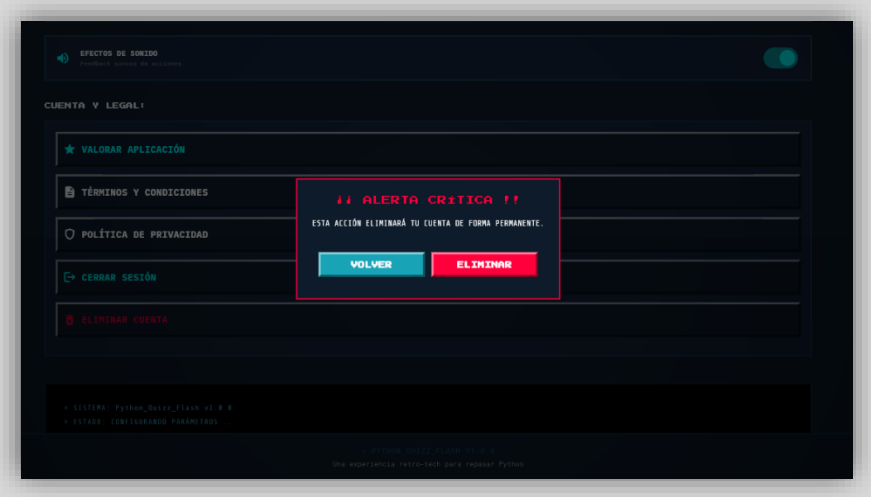
2. **Capa de Estado (Providers):** Clases como QuizProvider y AuthProvider que utilizan el patrón *Observer* (vía ChangeNotifier) para notificar cambios a la interfaz de usuario.



Primera aplicación completa en Flutter.-

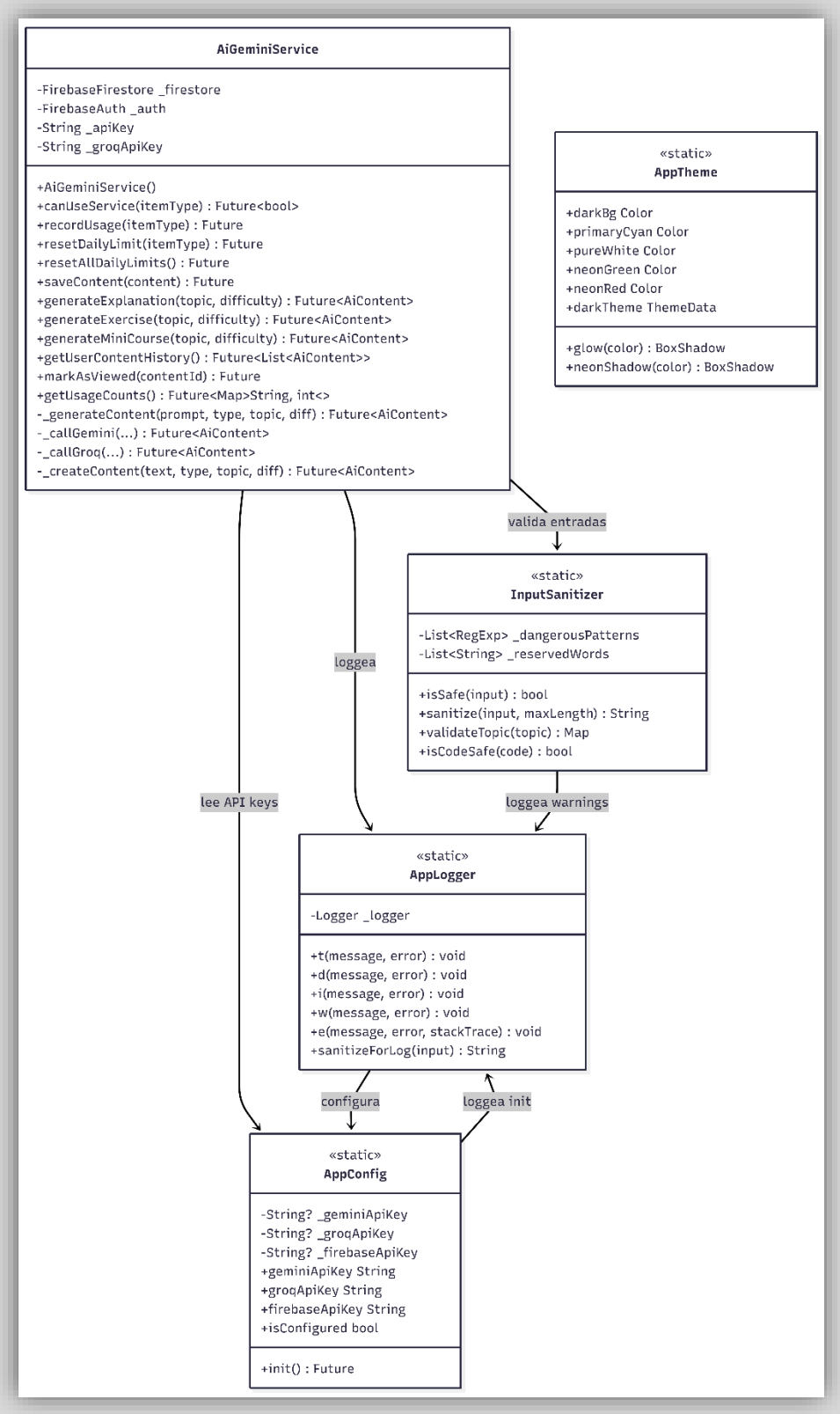
Este diagrama muestra los providers de Riverpod que gestionan el estado de la aplicación:

- **Auth**: “AsyncNotifierProvider” que maneja login, registro, logout y elimi nación de cuenta. Implementa rate limiting (5 intentos, bloqueo de 15 min).
- **Quiz**: “AsyncNotifierProviderFamily” parametrizado por “Difficulty”. Gestiona el estado completo del quiz: carga ejercicios desde JSON, valida respuestas, actualiza monedas y progreso.
- **LevelProgressNotifier**: “AsyncNotifierProvider” que rastrea el progreso por nivel (básico, intermedio, avanzado) y sincroniza con Firestore.
- **AppSettingsNotifier**: “NotifierProvider” para configuraciones de la app (música, SFX, vibración, notificaciones, idioma). Persiste en SecureSto rage.
- **EconomiaProvider**: “ChangeNotifier” singleton que gestiona la econo-mía del juego (PY-Coins, estadísticas). Sincronización Local (SharedPre ferences) y remoto (Firestore).
- **MusicController**: “NotifierProvider” con “keepAlive: true” para música de fondo usando “AudioPlayers”.
- **SecureStorageService**: Servicio auxiliar para almacenamiento seguro (configuraciones, tokens).



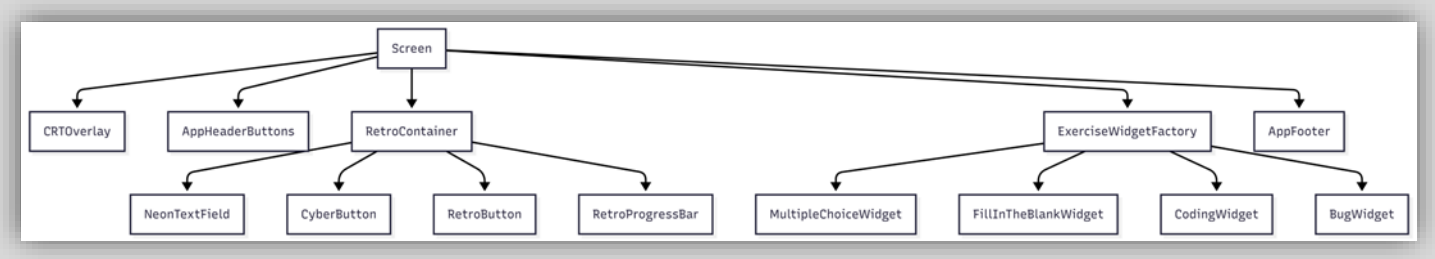
Primera aplicación completa en Flutter.-

3. **Servicios y Utilidades:** El AiGeminiService actúa como un cliente para la API de IA, abstrayendo la complejidad de las peticiones HTTP de la lógica de negocio.



- **AiGeminiService**: Servicio central de IA con fallback en cascada (Gemini 1.5 Pro → Gemini 2.0 Flash → Groq Llama 3). Implementa límites diarios de uso (1 por tipo/día), sanitización de inputs contra prompt injection, y persistencia de contenido en Firestore.
- **AppLogger**: Logger estático con niveles (trace, debug, info, warning, error). En producción solo activa warning/error. Incluye sanitización de datos sensibles (emails, API keys).
- **InputSanitizer**: Clase de seguridad con 35+ patrones regex para detec tar intentos de prompt injection en inglés y español. Valida temas (2-200 chars) y verifica seguridad de código Python.
- AppConfig: Configuración estática cargada desde ".env" (mobile/desk top) o "window.flutter_config" (web).
- AppTheme: Tema visual estático con 20+ colores, tema Material 3 dark completo, tipografías personalizadas (PressStart2P, IBMPlexMono, Fira Code) y efectos de neon glow.

4. **Widgets Personalizados:** siguen un sistema de diseño coherente con estética retro/cyberpunk



- **RetroButton**: Botón principal con 4 estilos, modo Win95, animaciones hover/press, iconos, y personalización completa.
- **CyberButton**: Botón cyber con efecto 3D de presión (bordes asimétricos que se invierten) y soporte para estado de carga.
- **NeonTextField**: Campo de texto con borde neon cyan, esquinas cuadra das, validación de formularios y modo password.
- RetroInput: Input estilo terminal con cursor parpadeante (500ms) y pre-fijo "> ".
- **RetroContainer**: Contenedor con bordes biselados, soporta estilos Win95 y cyberpunk neon.

Primera aplicación completa en Flutter.-



- **AppHeaderButtons**: Header reutilizable con botones VOLVER/SALIR con tamaños adaptativos.
- **PixelClip, PixelClipLogo, PixelClipDialog**: Mascota animada de la app con animaciones de levitación y glow.
- **ExerciseWidgetFactory**: Factory pattern que crea el widget apropiado según el tipo de ejercicio ("multipleChoice" → "MultipleChoiceWidget", etc.).

Tabla Completa de Widgets

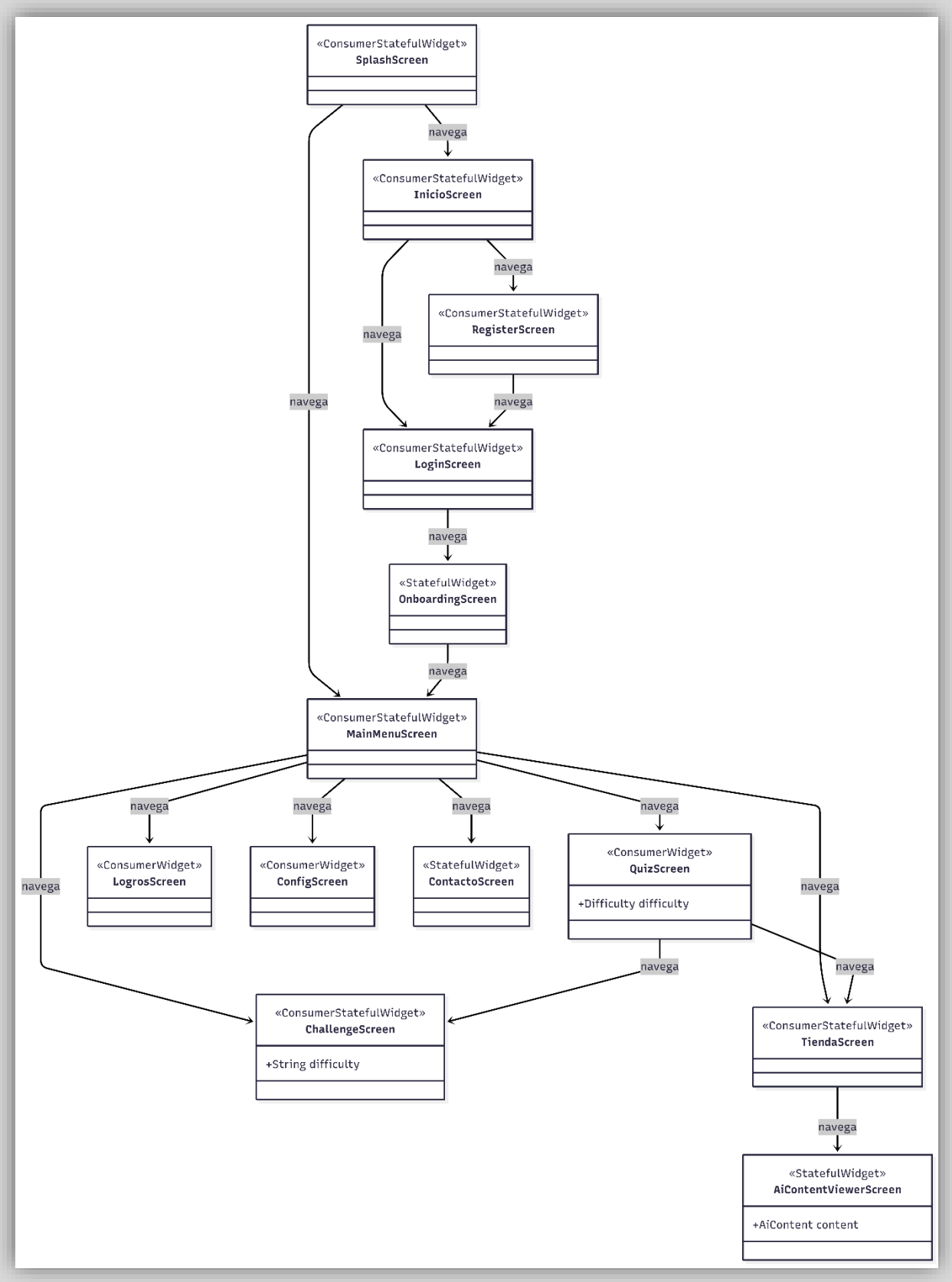
Widget	Archivo	Propósito	Props Principales	Efectos Visuales
CyberButton	cyber_button.dart	Botón principal con glow neón	text, onPressed, color	Sombra neón, hover effect, feedback táctil
RetroButton	retro_button.dart	Botón estilo Windows 95	text, onPressed, icon	Bordes 3D clásicos, colores Win95
NeonTextField	neon_text_field.dart	Input con borde luminoso	controller, hintText, obscureText	Glow en focus, borde animado
RetroContainer	retro_container.dart	Contenedor con marco retro	child, padding, color	Marco pixelado, fondo oscuro
RetroInput	retro_input.dart	Input estilo terminal retro	controller, hintText	Fuente monospace, cursor parpadeante
RetroProgressBar	retro_progress_bar.dart	Barra de progreso pixelada	progress, color, height	Segmentos pixelados, animación suave
CodingWidget	coding_widget.dart	Editor de código Python	onSubmit, initialCode, placeholder	Syntax highlighting, fuente FiraCode
CRTOverlay	crt_overlay.dart	Efecto CRT (scanlines)	child	Líneas de escaneo, viñeta, parpadeo
PixelClip	pixel_clip.dart	Clip de borde pixelado	child, borderRadius	Bordes escalonados estilo 8-bit

Primera aplicación completa en Flutter.-

PixelClipLogo	pixel_clip_logo.dart	Logo con recorte pixelado	size, color	Forma pixelada con glow
TypewriterText	typewriter_text.dart	Texto con efecto máquina de escribir	text, speed, style	Aparición letra por letra
BugWidget	bug_widget.dart	Visualización de detección de bugs	code, bugLine	Resaltado de línea con error
ChallengeDialog	challenge_dialog.dart	Diálogo de reto final	difficulty, onComplete	Temporizador, estilo alerta
AiPurchaseDialog	ai_purchase_dialog.dart	Diálogo de compra de contenido IA	itemType, topic, onConfirm	Preview del contenido, precio
AppHeaderButtons	app_header_buttons.dart	Botones de navegación en header	N/A	Iconos de logros, tienda, config
AppFooter	app_footer.dart	Pie de página con créditos	showMusicToggle	Links legales, toggle música
AppNotifications	app_notifications.dart	Sistema de notificaciones toast	N/A	Snackbar con estilo cyberpunk
ExerciseWidgetFactory	exercise_widget_factory.dart	Factory que retorna widget según tipo de ejercicio	exercise, onAnswer	Adapta UI al tipo (multipleChoice, fillBlank, coding, bug)
FillInTheBlankWidget	fill_in_the_blank_widget.dart	Widget de completar huecos	exercise, onSubmit	Input inline con código
MultipleChoiceWidget	multiple_choice_widget.dart	Widget de opción múltiple	exercise, onSelect, disabledOptions	Botones con opciones A/B/C/D

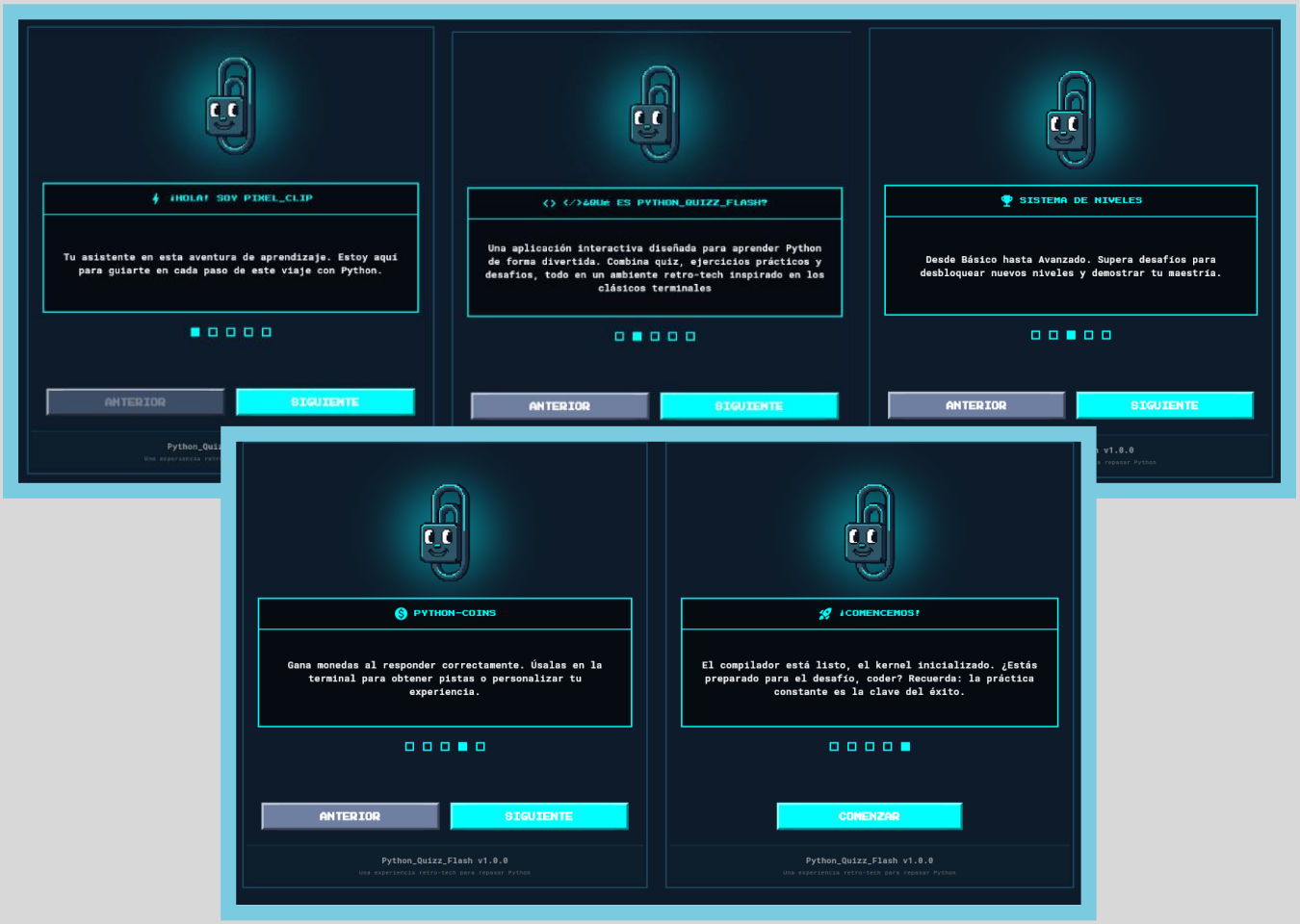
Primera aplicación completa en Flutter.-

5. **Pantallas Principales**: siguen una jerarquía de navegación clara.



Primera aplicación completa en Flutter.-

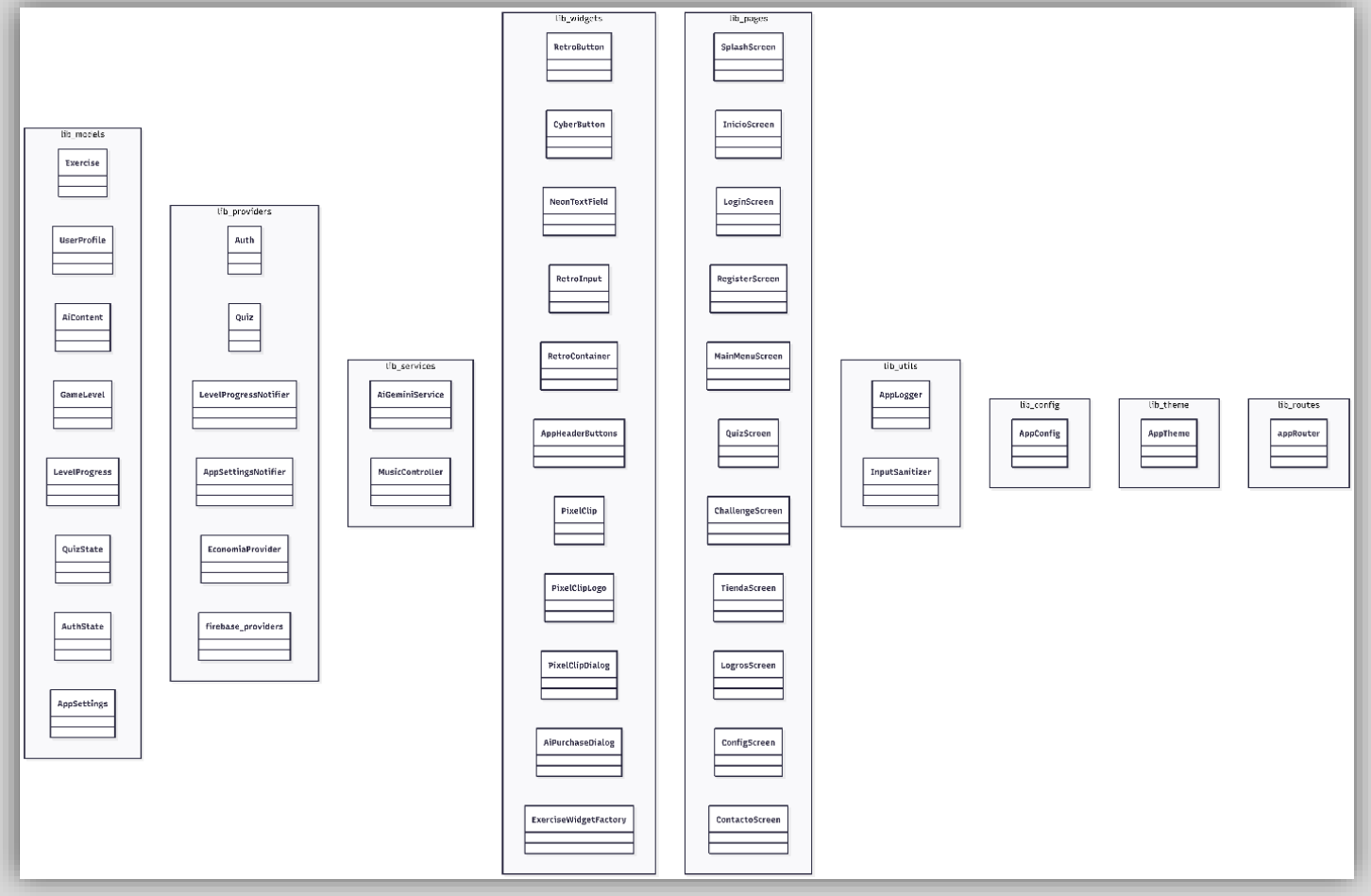
- **SplashScreen** → Punto de entrada, decide entre autenticado (“MainMe nuScreen”) o no (“InicioScreen”).
- **InicioScreen** → Landing page con opciones de login/registro.
- **LoginScreen/RegisterScreen** → Flujo de autenticación que lleva a “OnboardingScreen”.
- **OnboardingScreen** → Carousel tutorial de 5 slides que lleva al menú principal.
- **MainMenuScreen** → Hub central de navegación a todas las funcionali dades.
- **QuizScreen/ChallengeScreen** → Pantallas de juego interconectadas.
- **TiendaScreen** → AiContentViewerScreen → Flujo de compra de conte nido IA.
- **LogrosScreen/ConfigScreen/ContactoScreen** → Pantallas terminales (no navegan a otras).



Primera aplicación completa en Flutter.-

Diagrama de Paquetes

La organización del código sigue una arquitectura por características (feature-first).

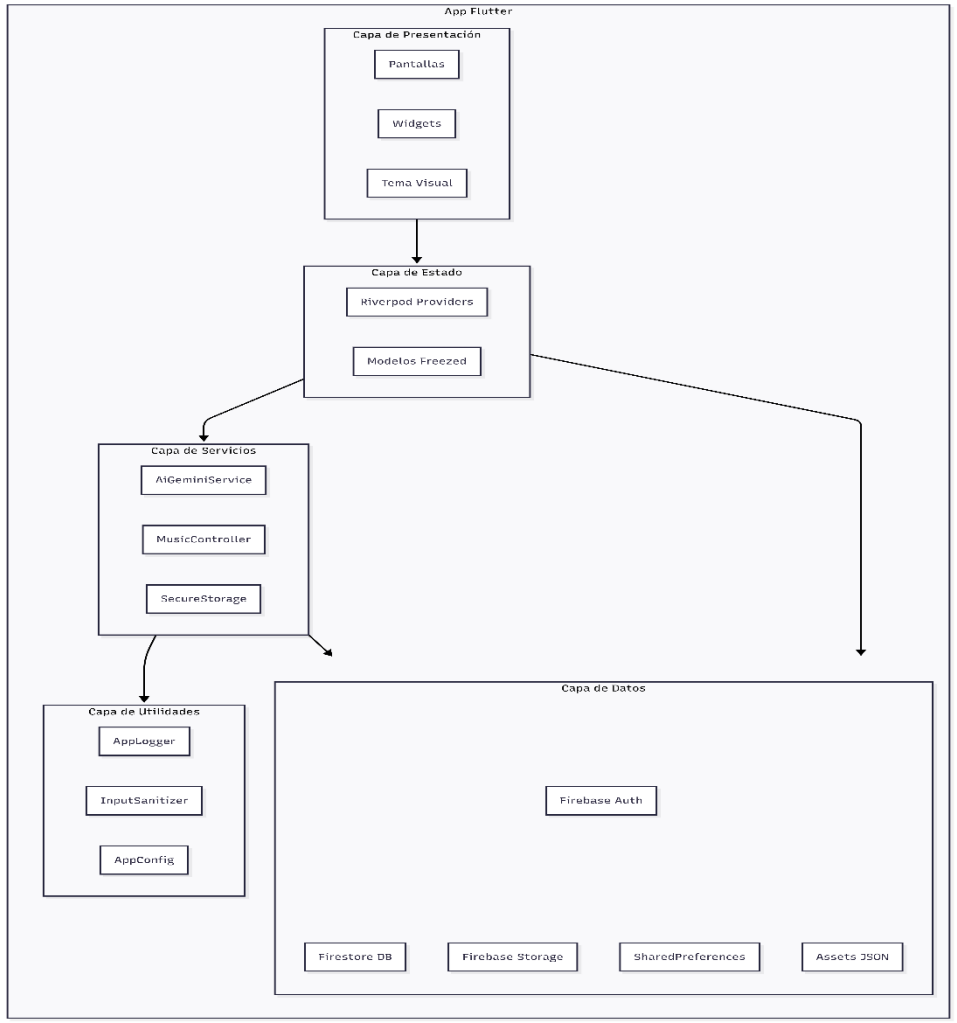


- **models**: Entidades de datos puras, inmutables, la mayoría con Freezed.
- **providers**: Lógica de estado y negocio con Riverpod.
- **services**: Integraciones externas (IA, música).
- **widgets**: Componentes UI reutilizables.
- **pages**: Pantallas completas de la aplicación.
- **utils**: Utilidades transversales (logging, seguridad).
- **config/theme/routes**: Configuración global de la app.

Primera aplicación completa en Flutter.-

Diagrama de Componentes

Muestra la estructura física o modular (paquetes, archivos, librerías)



- **Capa de Presentación**: Pantallas (Views) + Widgets reutilizables (Componentes) + Tema visual.
- **Capa de Estado**: Providers de Riverpod (ViewModels) + Modelos Freezed inmutables.
- **Capa de Servicios**: IA (Gemini/Groq), música, almacenamiento seguro.
- **Capa de Datos**: Firebase (Auth, Firestore, Storage) + SharedPreferences + Assets JSON.
- **Capa de Utilidades**: Logging, sanitización, configuración.

Diagrama de Secuencia - Flujo de Autenticación

Se enfoca en el orden cronológico de los mensajes que se intercambian entre diferentes objetos o componentes para completar una tarea, en este caso, el proceso de inicio de sesión.

Explicación paso a paso:

1) El usuario ingresa sus credenciales en “LoginScreen”.

2) “LoginScreen” valida que los campos no estén vacíos y llama a “Auth.login()”.

3) Rate limiting: “Auth” verifica si el identifier ha superado 5 intentos fallidos en los últimos 15 minutos. Si es así, rechaza el intento.

4) Si pasa el rate limiting, intenta autenticar con Firebase Auth.

5) Si las credenciales son incorrectas, incrementa el contador de intentos fallidos y muestra error.

6) Si el usuario no existe directamente, busca por “username_search” en Firestore (la app permite login por email o username).

7) Si encuentra por username, obtiene el email asociado, autentica, carga el perfil y la economía, luego navega a “/onboarding”.

8) Si el login es exitoso directamente, carga perfil y economía, navega a “/main_menu”.

Primera aplicación completa en Flutter.-

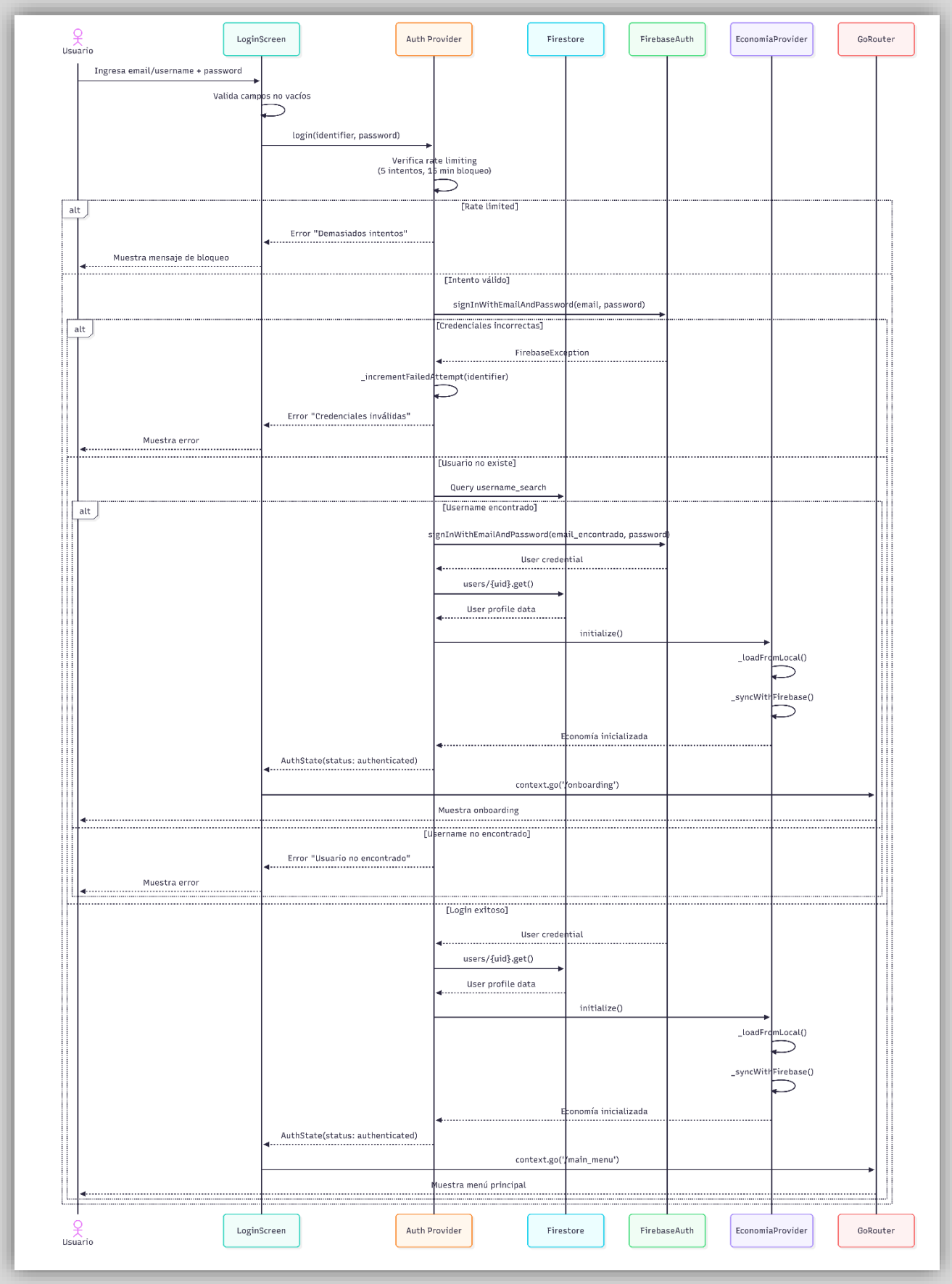


Diagrama de Secuencia - Flujo del Quiz

Explicación paso a paso:

1) El usuario inicia un nivel desde “MainMenuScreen”.

2) GoRouter navega a “QuizScreen” con el parámetro “difficulty”.

3) “Quiz.build()” carga los ejercicios desde el JSON local, filtra por dificultad, selecciona 20 aleatorios + 1 challenge.

4) Restaura el progreso guardado en Firestore (índice actual, monedas, errores).

5) “ExerciseWidgetFactory” crea el widget apropiado según el tipo de ejercicio.

6) Por cada pregunta: el usuario responde → se valida → se actualiza economía y Firestore → se muestra feedback.

7) Al completar las 20 preguntas: si errores >= 5, muestra pantalla de error; si < 5, muestra éxito y ofrece ir al challenge.

Primera aplicación completa en Flutter.-

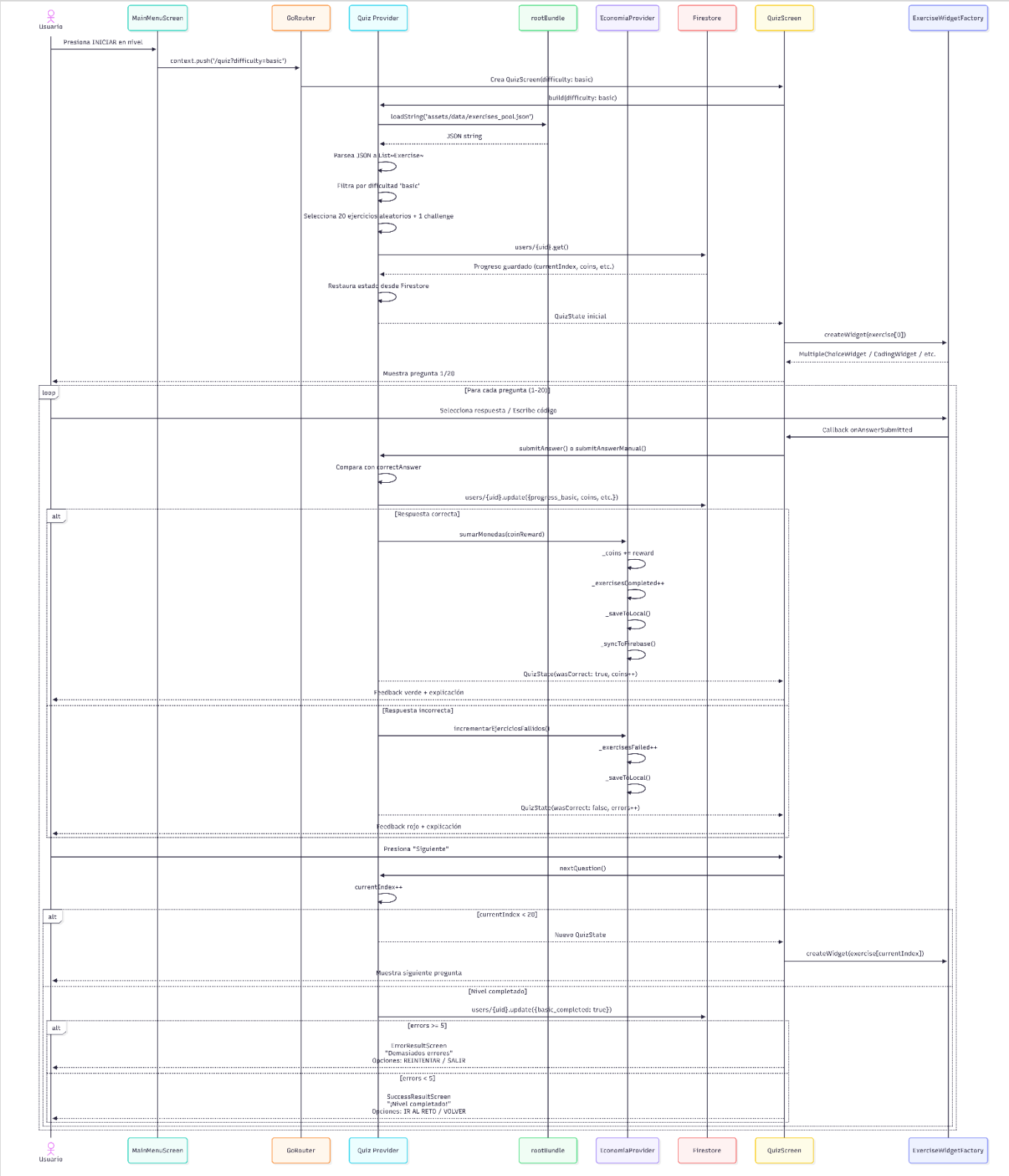


Diagrama de Secuencia - Compra de Contenido IA

Explicación paso a paso:

1) El usuario accede a la tienda desde el menú principal.

2) Selecciona un tipo de contenido (explicación, ejercicio, mini-curso).

3) Selección de tema: Se muestra un diálogo para ingresar el tema. El tema se valida con “InputSanitizer” (longitud + detección de prompt injection).

4) Diálogo de compra: Se confirma el item y precio. Si el usuario confirma: Verificación de monedas: “EconomiaProvider” descuenta las monedas. Si no hay suficientes, muestra error.

5) Verificación de límite diario: “AiGeminiService” consulta “ai_usage” en Firestore. Si ya usó ese tipo hoy, rechaza.

6) Generación con fallback: Intenta Gemini 1.5 Pro → si falla, Gemini 2.0 Flash → si falla, Groq Llama 3.

7) Guardado: El contenido se guarda en “ai_content” y se registra el uso en “ai_usage”.

8) Visualización: Navega a “AiContentViewerScreen” que muestra el contenido formateado con código extraíble.

Primera aplicación completa en Flutter.-

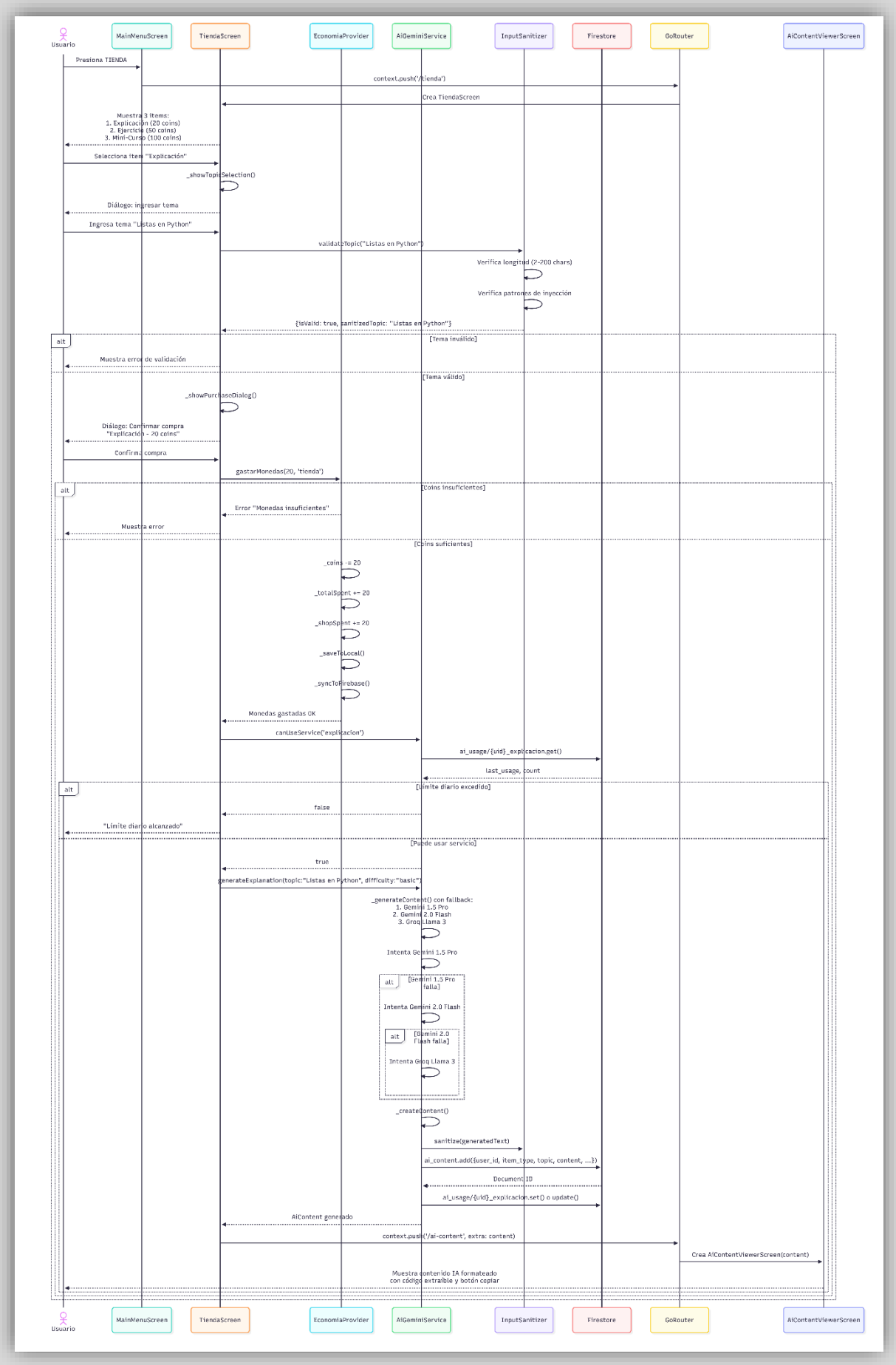
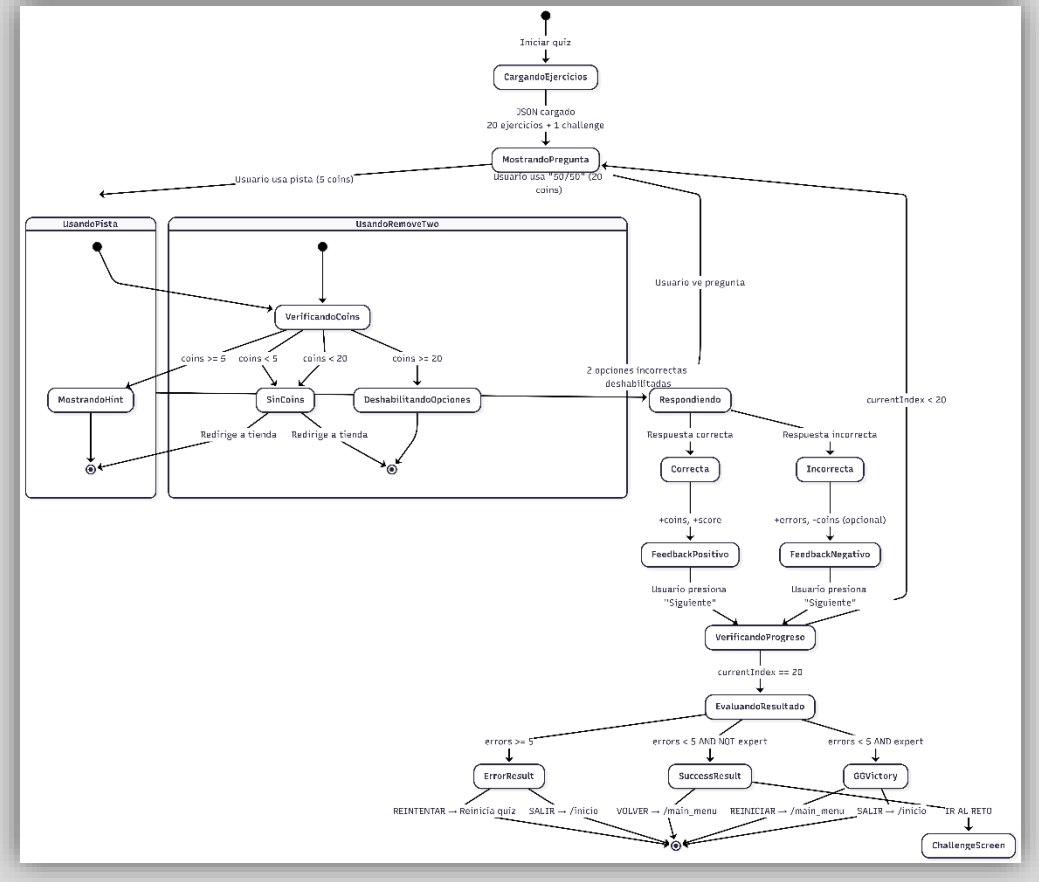


Diagrama de Estados

Describe el comportamiento de un sistema o un objeto específico al mostrar los diferentes "momentos" o condiciones por las que pasa durante su vida, y qué eventos provocan un cambio entre ellos.

Ciclo de Vida del Quiz

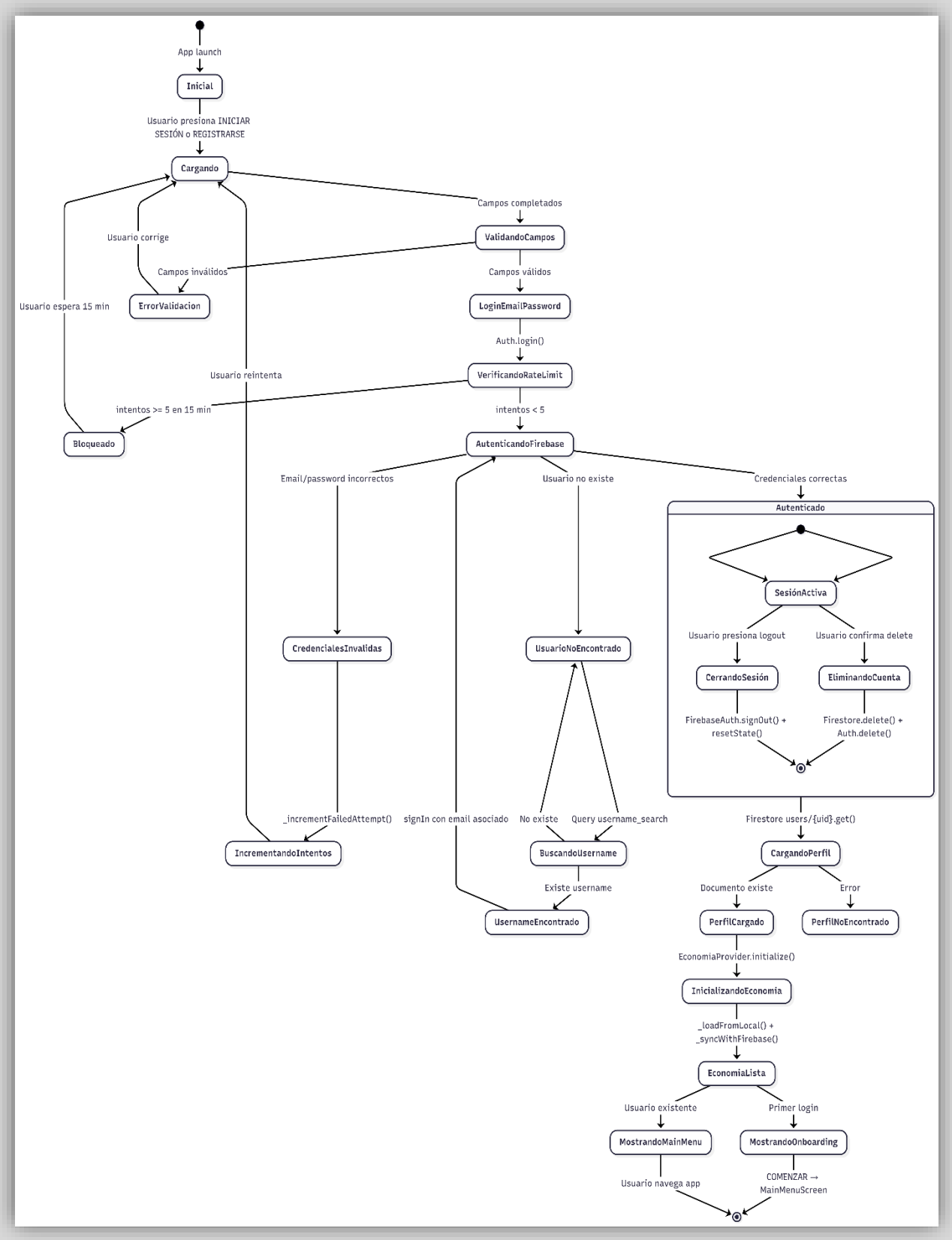


La máquina de estados del quiz sigue este flujo:

1) CargandoEjercicios: Carga desde JSON local, filtra por dificultad, selecciona 20 aleatorios.
2) MostrandoPregunta → Respondiendo: El usuario interactúa con el widget de ejercicio.
3) Respondiendo → Correcta/Incorrecta: Se valida y actualiza estado.
4) Feedback: Muestra explicación (verde si correcto, rojo si incorrecto).
5) VerificandoProgreso: Si quedan preguntas, continúa; si no, evalúa resultado.
6) EvaluandoResultado: Decide entre ErrorResult (>=5 errores), SuccessResult (<5 errores), o GGVictory (expert completado).
7) Power-ups: Durante la pregunta, el usuario puede usar pistas (5 coins) o 50/50 (20 coins).

Primera aplicación completa en Flutter.-

Máquina de Estados de Autenticación



La máquina de estados de autenticación maneja:

1) Inicial → Cargando: Usuario inicia flujo de login/registro.
2) ValidandoCampos: Verifica que email, password, username cumplan requisitos.
3) VerificandoRateLimit: Comprueba si hay bloqueo por intentos fallidos.
4) AutenticandoFirebase: Intenta login con Firebase Auth. Si falla, incrementa contador.
5) BuscandoUsername: Si el login directo falla, busca por "username_search" en Firestore (permite login por username).
6) CargandoPerfil → InicializandoEconomia: Carga datos del usuario y sincroniza economía local/remota.
7) MostrandoOnboarding/Menu: Decide según si es primer login o usuario existente.
8) Sub-estados de Autenticado: Sesión activa puede terminar en logout o eliminación de cuenta.

Diagrama de Despliegue

El diagrama de despliegue muestra la arquitectura física de la aplicación:

Lado del Cliente (Dispositivo)

- App Flutter: Multiplataforma (iOS, Android, Web, Windows, Linux, macOS).
- Capa UI: Widgets y pantallas renderizadas por Skia/Impeller.
- Capa Estado: Riverpod providers gestionando estado en memoria.
- Almacenamiento Local: SharedPreferences (economía, settings) + SecureStorage (configuraciones sensibles).

Lado del Servidor (Firebase Cloud)

- Firebase Authentication: Gestión de usuarios (registro, login, session persistence).
- Firestore Database: 4 colecciones ("users", "transactions", "ai_content", "ai_usage").
- Firebase Storage: Archivos de usuario (privado) y públicos.

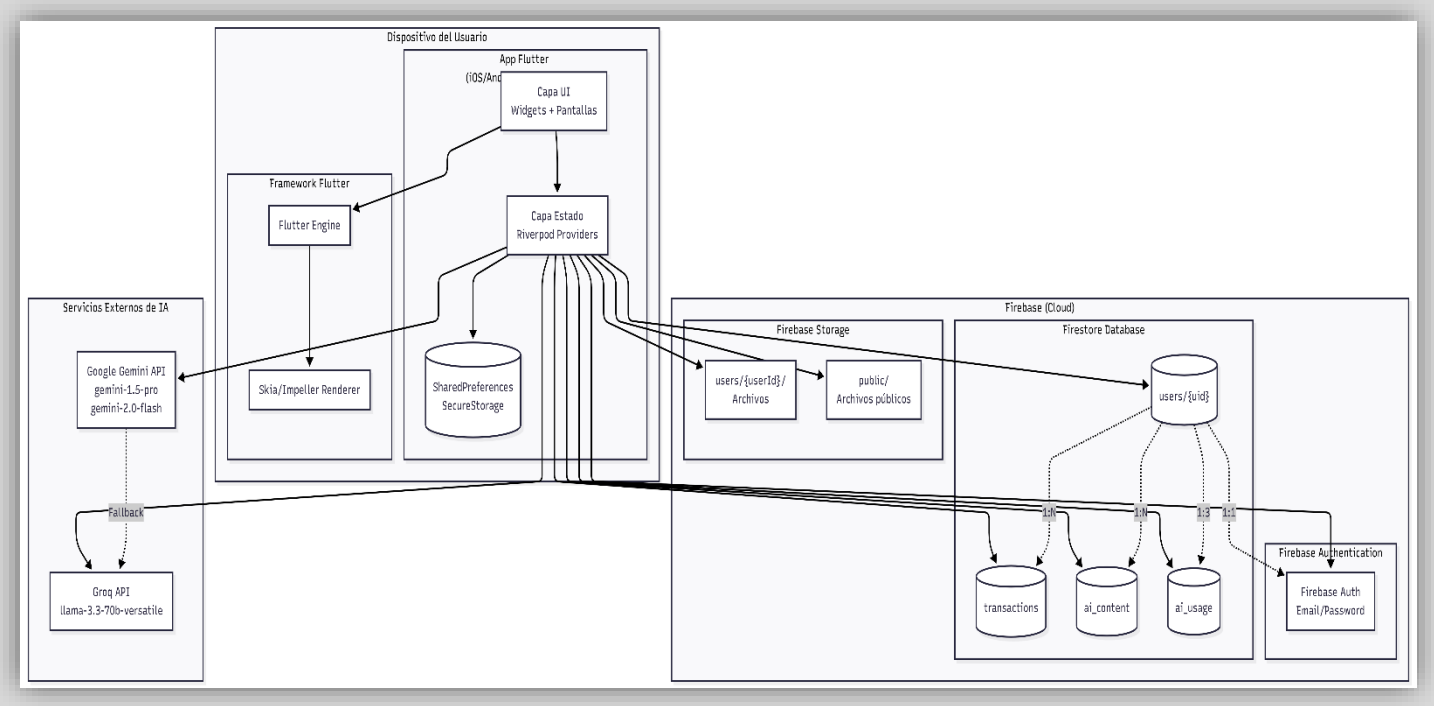
Primera aplicación completa en Flutter.-

Servicios Externos

- Google Gemini API: IA primaria con 2 modelos (1.5 Pro principal, 2.0 Flash fallback).
- Groq API: IA secundaria de fallback (Llama 3 70B).

Flujo de Datos

- El usuario interactúa con la UI → actualiza estado via Riverpod.
- El estado persiste localmente (SharedPreferences) y remotamente (Firestore).
- Para contenido IA, se hace fallback en cascada: Gemini 1.5 Pro → Gemini 2.0 Flash → Groq.
- Firebase Auth mantiene persistencia de sesión en el navegador (Persistence.LOCAL).



Decisiones de Diseño

- ¿Por qué Freezed para los modelos?

  Freezed genera automáticamente “copyWith”, “==“, “hashCode”, “toString”, y pattern mat ching para clases inmutables. Esto reduce bugs por mutaciones accidentales y boilerplate. En una app de quiz donde el estado cambia frecuentemente (respuestas, progreso, mone das), la inmutabilidad garantiza que cada cambio de estado crea un nuevo objeto, facili tando la trazabilidad y el debugging.

- ¿Por qué Riverpod en lugar de Provider/BLoC?

  Riverpod ofrece:

  - Seguridad en tiempo de compilación: Los providers no pueden ser leídos sin ser declarados.
  - Family providers: “QuizProvider” parametrizado por “Difficulty” sin crear múlti- ples instancias manualmente.
  - AsyncNotifier: Manejo nativo de estados asíncronos (carga de Firestore, vali- dación de login).
  - keepAlive: “MusicController” sobrevive aunque no se use (la música no debe cortarse al cambiar de pantalla).
  - Mejor testing: Los providers son inyectables y mockeables.

- ¿Por qué fallback en cascada para IA?Gemini 1.5 Pro puede tener rate limits o caídas. Tener Gemini 2.0 Flash como fallback inmediato y Groq Llama 3 como último recurso garantiza que el usuario siempre reciba contenido IA, mejorando la experiencia de usuario y redu- ciendo errores en producción.

- ¿Por qué SharedPreferences + Firestore para economía?

  SharedPreferences permite acceso offline inmediato (mejor UX), mientras Firestore sincro niza para multi-dispositivo y backup. La lógica de sync toma el valor más alto entre local y remoto, evitando pérdida de progreso.

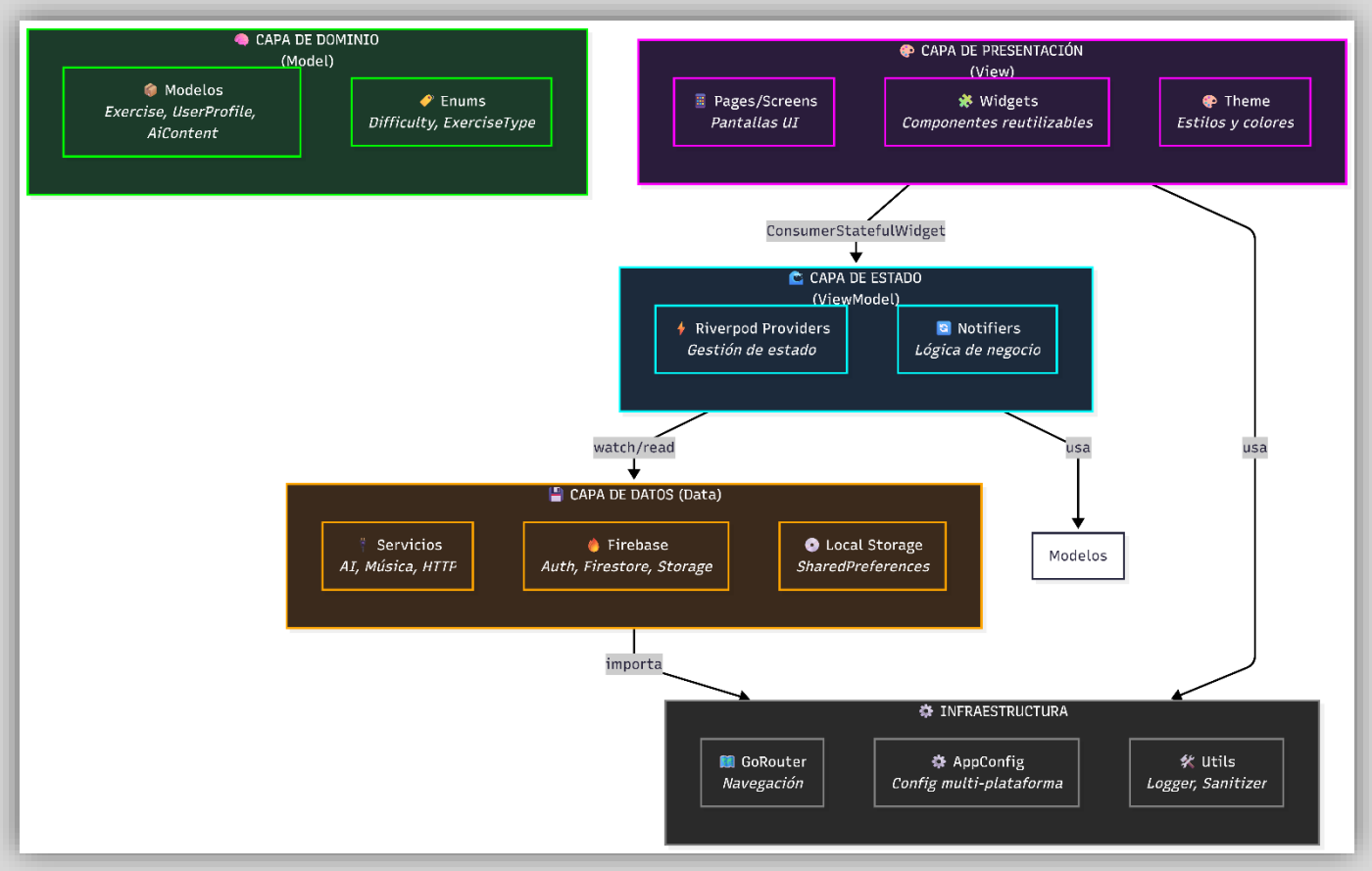
- ¿Por qué lectura pública de “users” pero escritura restringida?La app permite login por user- name (no solo email). Para buscar un username, se debe consultar Firestore antes de au- tenticar. Por eso “users” permite lectura pública. Pero la escritura está restringida al dueño del documento para seguridad.

Arquitectura de Software , Patrones de Diseño y Modelo de Datos

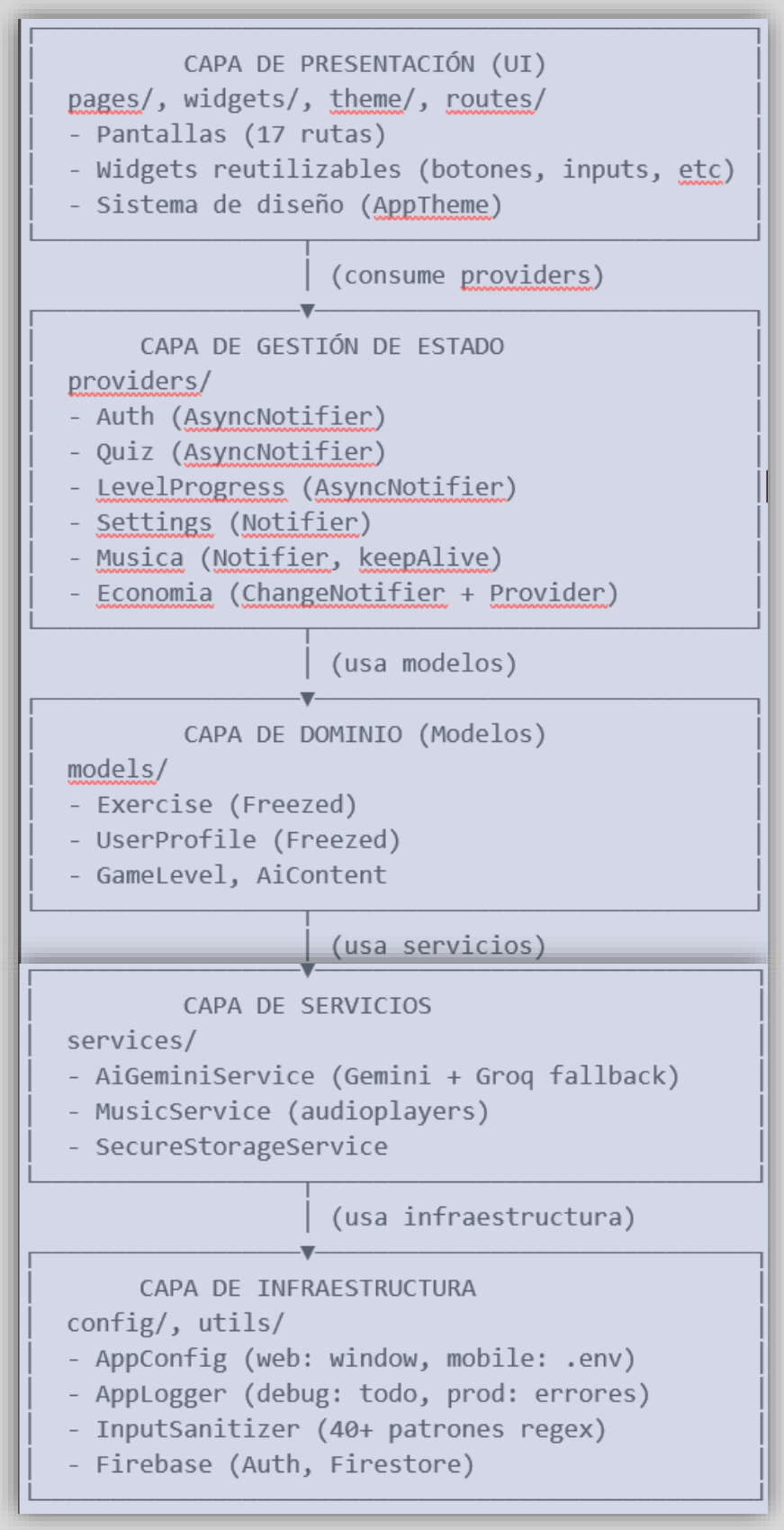
Arquitectura de Capas (Clean Architecture)

El proyecto no es solo un conjunto de pantallas; es un ecosistema desacoplado. Se ha implementado una arquitectura de **5 capas** para garantizar que un cambio en la base de datos no rompa la interfaz:

1. **Presentación:** Widgets reactivos de Flutter.
2. **Estado:** Lógica de negocio gestionada por **Riverpod**.
3. **Dominio:** Modelos de datos inmutables (usando Freezed).
4. **Datos y Servicios:** AIs ,Firebase, Local Storage.
5. **Infraestructura:** Implementación real de los datos.



Primera aplicación completa en Flutter.-



Primera aplicación completa en Flutter.-

Estructura del Proyecto

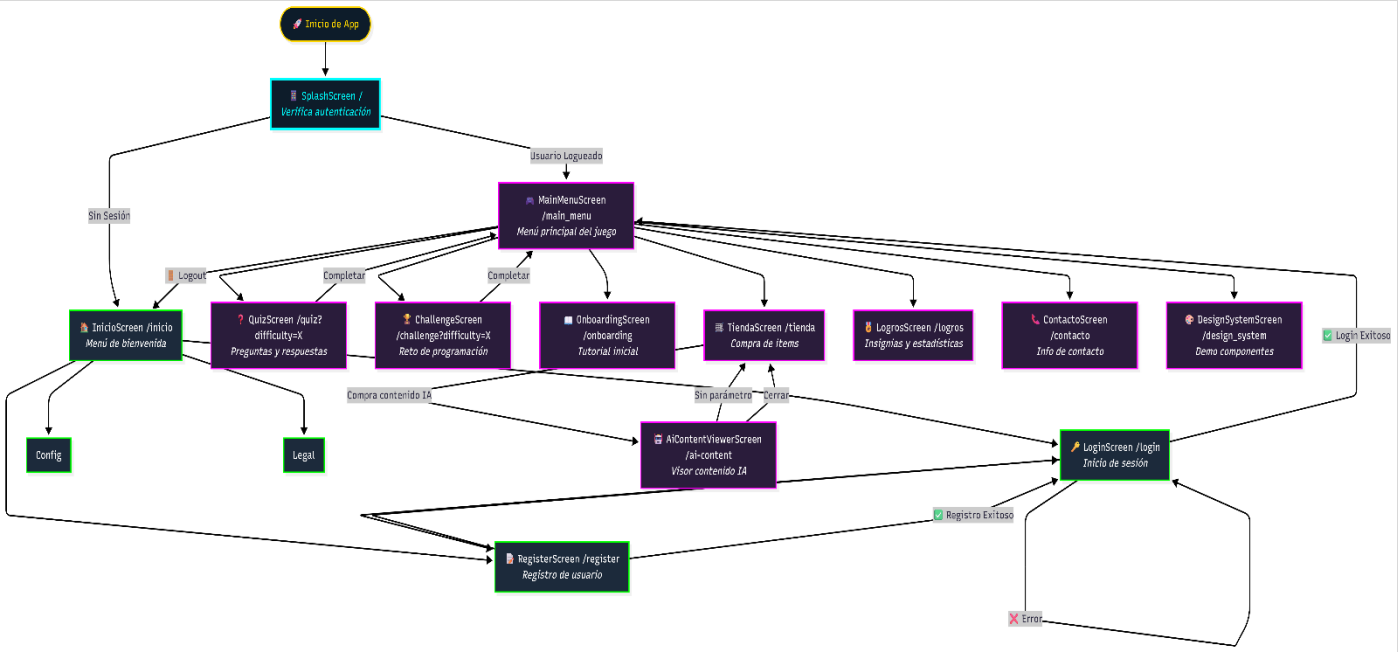
```
python_quizz_flash/
├── lib/
│   ├── main.dart                        # Punto de entrada
│   ├── config/
│   │   └── app_config.dart              # Configuración unificada
│   ├── models/
│   │   ├── ai_content.dart              # Modelo de contenido IA
│   │   ├── exercise.dart                # Modelo de ejercicios
│   │   ├── game_level.dart              # Modelo de niveles
│   │   └── user_profile.dart            # Modelo de usuario
│   ├── pages/
│   │   ├── splash_screen.dart           # Pantalla de carga
│   │   ├── login_screen.dart            # Login
│   │   ├── register_screen.dart         # Registro
│   │   ├── main_menu_screen.dart        # Menú principal
│   │   ├── inicio_screen.dart           # Inicio
│   │   ├── tienda_screen.dart           # Tienda IA
│   │   ├── quiz_screen.dart             # Quiz
│   │   ├── challenge_screen.dart        # Desafíos
│   │   ├── config_screen.dart           # Configuración
│   │   ├── ai_content_viewer_screen.dart  # Visor de contenido IA
│   │   └── ...
│   ├── providers/
│   │   ├── auth_provider.dart           # Estado de autenticación
│   │   ├── quiz_provider.dart           # Estado del quiz
│   │   ├── settings_provider.dart       # Configuración de app
│   │   ├── level_progress_provider.dart   # Progreso de nivel
│   │   └── firebase_providers.dart      # Providers de Firebase
│   ├── routes/
│   │   └── app_router.dart              # Configuración de rutas (GoRouter)
│   ├── services/
│   │   ├── ai_gemini_service.dart       # Servicio de IA (Gemini/Groq)
│   │   └── music_service.dart           # Servicio de música
│   ├── theme/
│   │   └── app_theme.dart               # Tema y diseño
│   ├── utils/
│   │   ├── app_logger.dart              # Logger seguro
│   │   └── input_sanitizer.dart         # Sanitización de inputs
│   └── widgets/
│       ├── app_header_buttons.dart      # Botones de navegación
│       ├── app_footer.dart              # Pie de página
│       ├── cyber_button.dart            # Botón estilo cyber
│       ├── neon_text_field.dart         # Campo de texto neón
│       ├── retro_button.dart            # Botón retro
│       ├── retro_input.dart             # Input retro
│       ├── coding_widget.dart           # Widget de código
│       ├── ai_purchase_dialog.dart      # Diálogo de compra
│       └── ...
```

Primera aplicación completa en Flutter.-

```
├── web/
│   ├── index.html                    # HTML base para web
│   ├── config.js                     # Configuración para web
│   └── manifest.json                 # PWA manifest
├── android/                          # Configuración Android
├── ios/                              # Configuración iOS
├── assets/
│   ├── fonts/                        # Fuentes personalizadas
│   ├── images/                       # Imágenes y assets
│   ├── audio/music/                  # Música de fondo
│   └── data/                         # Datos estáticos
├── firestore.rules                   # Reglas de seguridad Firestore
├── storage.rules                     # Reglas de seguridad Storage
├── firebase.json                     # Configuración Firebase
└── pubspec.yaml                      # Dependencias del proyecto
```

Mapa de Rutas - Sistema de Navegación

El sistema de navegación de Python Quizz Flash está implementado utilizando Go Router, un sistema de enrutamiento declarativo para Flutter que permite navegación basada en URLs con soporte para parámetros, datos extra y redirección condicional. La configuración centralizada se encuentra en “lib/routes/app_router.dart”, donde se definen todas las rutas disponibles y sus respectivas condiciones de acceso.



Rutas Públicas (Sin Autenticación)

Estas pantallas son accesibles sin necesidad de tener un usuario autenticado:

Ruta	*Pantalla*	*Descripción*
"/"	"SplashScreen"	Pantalla de carga inicial que verifica sesión y navega automáticamente
"/inicio"	"InicioScreen"	Menú principal con diseño pixel art y botones de login/registro
"/login"	"LoginScreen"	Formulario de inicio de sesión con email/nick y contraseña
"/register"	"RegisterScreen"	Formulario de registro con validaciones de usuario, email y contraseña

Rutas Privadas (Protegidas - Requieren AuthProvider)

Estas pantallas requieren que el "AuthProvider" confirme un usuario activo:

Ruta	*Pantalla*	*Descripción*
"/main_menu"	"MainMenuScreen"	Menú principal del juego con niveles, monedas y navegación a secciones
"/quiz"	"QuizScreen"	Pantalla de preguntas/respuestas del quiz (recibe parámetro "difficulty")
"/challenge"	"ChallengeScreen"	Pantalla de reto final de programación (recibe parámetro "difficulty")
"/onboarding"	"OnboardingScreen"	Tutorial inicial con diapositivas explicativas
"/tienda"	"TiendaScreen"	Tienda de items, pistas y contenido generado por IA
"/logros"	"LogrosScreen"	Insignias, estadísticas e historial de transacciones
"/contacto"	"ContactoScreen"	Información de contacto y redes sociales del desarrollador
"/ai-content"	"AiContentViewerScreen"	Visor de contenido generado por IA (recibe objeto "AiContent")
"/design_system"	"DesignSystemScreen"	Pantalla de demostración de componentes del sistema de diseño

Archivos Relacionados

“lib/routes/app_router.dart” - Configuración central de rutas
“lib/main.dart” - Inicialización de MaterialApp.router
“lib/providers/auth_provider.dart” - Lógica de autenticación
“lib/providers/firebase_providers.dart” - StreamProvider de authState
“lib/pages/splash_screen.dart” - Lógica de verificación y navegación automática

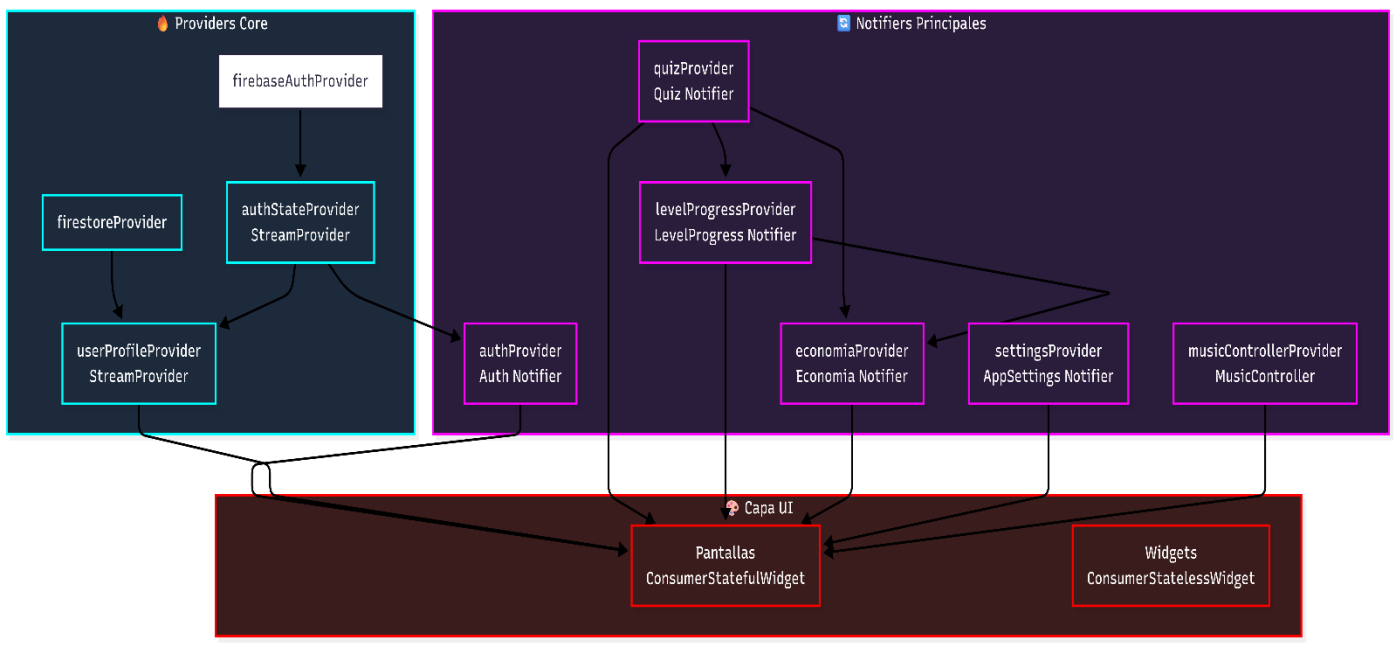
Resumen de Navegación

Característica	***Implementación***
Sistema de rutas	GoRouter v17.1.0
Rutas totales	17 rutas definidas
Rutas públicas	8 rutas (splash, inicio, login, register)
Rutas privadas	9 rutas (main_menu, quiz, challenge, tienda, logros, etc.)
Parámetros de ruta	3 rutas con parámetros (“/quiz”, “/challenge”, “/ai-content”)
Tipo de parámetros	Query parameters (difficulty) y datos extra (AiContent)
Verificación de auth	SplashScreen + AuthProvider (StreamProvider)
Persistencia de sesión	“Persistence.LOCAL” en Firebase Auth
Método de navegación	“context.go()” (reemplaza) y “context.push()” (apila)

Gestión de Estado con Riverpod

A diferencia de *Provider*, Riverpod nos permite capturar errores en tiempo de compila ción. Usamos **6 Providers clave** (Auth, Quiz, Economía, etc.) que funcionan como una "fuente de verdad única".

El uso de Riverpod Generator reduce el código manual (boilerplate) y garantiza que los estados se destruyan cuando no se usan, optimizando la memoria del dispositivo,algo crítico en dispositivos móviles de gama media.



Patrones de Diseño Utilizados

Provider Pattern (Riverpod)

Uso: Gestión de estado global y inyección de dependencias.

```
// Definición
@riverpod
class Auth extends \_$Auth {
 AuthState build() => const AuthState();
}

// Consumo en UI
final authState = ref.watch(authProvider);
ref.read(authProvider.notifier).login(...);
```

Primera aplicación completa en Flutter.-

Factory Pattern

Uso: Creación de objetos complejos (Freezed models, estados).

```
@freezed
abstract class AuthState with \_$AuthState {
const factory AuthState({
@Default(AuthStatus.initial) AuthStatus status,
@Default('SISTEMA\_EN\_ESPERA...') String message,
 }) = \_AuthState;
}
```

Strategy Pattern

Uso: Diferentes tipos de ejercicios con comportamiento común.

```
enum ExerciseType { multipleChoice, fillInTheBlank, bugFix, coding }

// Widget factory selecciona estrategia según tipo
ExerciseWidgetFactory.createExercise(exercise);
```

State Pattern

Uso: Estados de autenticación y quiz con transiciones claras.

```
enum AuthStatus { initial, loading, authenticated, error, success }

state = state.copyWith(status: AuthStatus.loading);
```

Singleton Pattern

Uso: Servicios globales (Firebase instances, AppConfig).

```
class AppConfig {
static String? \_geminiApiKey;
static Future<void> init() async { ... }
 static String get geminiApiKey => \_geminiApiKey ?? '';
}
```

Gestión de Datos e Integración de Inteligencia Artificial

Modelo de Datos y Persistencia (ERD)

Se utiliza un enfoque híbrido de persistencia: Firebase Firestore para datos en tiempo real y sincronización en la nube, y Assets JSON para contenido estático (pool de ejercicios). Esto optimiza el rendimiento y reduce los costos de lectura de API al no tener que descargar cientos de ejercicios cada vez.

Almacenamiento Estático Local (Assets JSON)

Aunque la aplicación utiliza Firebase para los datos dinámicos del usuario, el pool de ejercicios principal se gestiona de forma local.

Se ha optado por un almacenamiento local en formato JSON para los más de 150 ejercicios por tres razones estratégicas:

- Rendimiento (Baja latencia): El acceso a los ejercicios es instantáneo, eliminando tiempos de carga de red al iniciar un quiz.
- Reducción de Costes: Al no realizar peticiones de lectura a Firestore por cada ejercicio, se optimiza el uso de la cuota gratuita (Spark Plan) del backend.
- Disponibilidad Offline: Permite que la lógica educativa de la aplicación funcione incluso sin conexión a internet, delegando a Firebase solo la sincronización final de puntos.
- Economía de Servidor: Se evitan miles de lecturas a Firebase, manteniéndose en el Free Tier.
- Modo Offline: La app puede ser funcional pedagógicamente sin internet.

Los ejercicios se almacenan en “assets/data/exercises_pool.json “ con esta estructura:

```
{
  "basic": [
    {
      "id": "b_1",
      "difficulty": "basic",
      "type": "multipleChoice",
      "question": "¿Cuál es el tipo de dato de x = 5?",
      "options": ["str", "int", "float", "bool"],
      "correctAnswer": "int",
      "defaultExplanation": "Los números sin decimales son int.",
      "hint": "Es un número sin decimales.",
      "coinReward": 10,
      "isChallenge": false
    }
  ],
  "intermediate": [...],
  "advanced": [...],
  "expert": [...]
}
```

Modelado y Serialización

Para consumir estos datos en Flutter, se utiliza el paquete json_serializable y modelos inmutables con Freezed. El flujo es:

- Carga del archivo mediante rootBundle.loadString().
- Mapeo del JSON a una lista de objetos ExerciseModel.
- Inyección mediante un Provider de Riverpod para que los datos estén disponibles en toda la interfaz de forma eficiente.

Modelo de Datos y persistencia en Cloud Firestore (NoSQL)

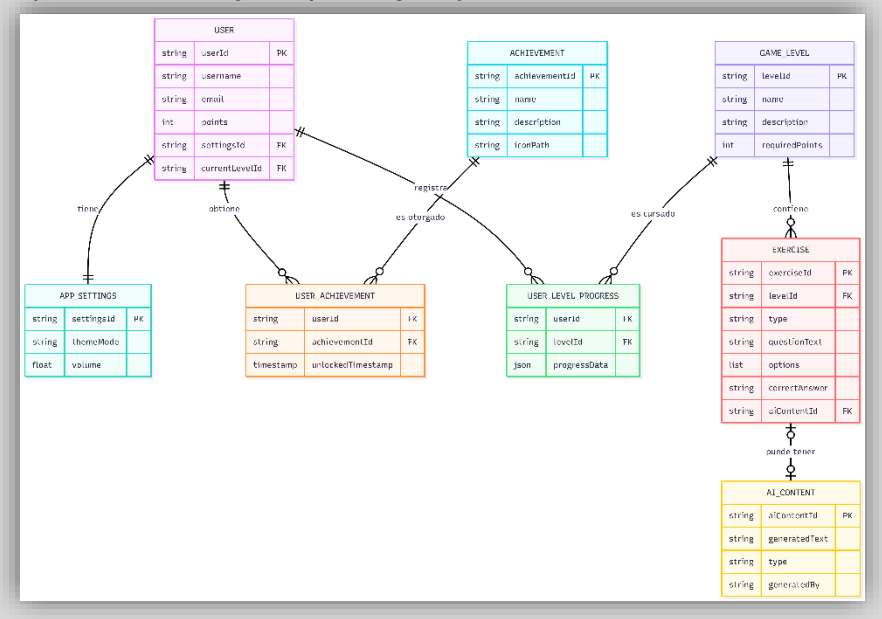
Se utiliza una base de datos NoSQL ya que está diseñada para ser ultra-rápida. A diferencia de un modelo relacional tradicional, aquí los datos se organizan en Colec ciones de documentos independientes:

- Colección users: Cada documento usa el UID de Firebase Auth como identificador único. Almacena de forma desnormalizada el progreso (progress_basic, level), las estadísticas de aciertos/errores y el saldo de "PY-Coins".

- Colección ai_content: Almacena las explicaciones generadas por la API de Gemini. Cada documento incluye una referencia (user_id) para vincular el contenido con su creador.

- Colección transactions: Un registro inmutable de la economía del usuario (compras en la tienda y gastos en pistas).

Diagrama para ilustrar la jerarquía lógica y las referencias entre documentos.



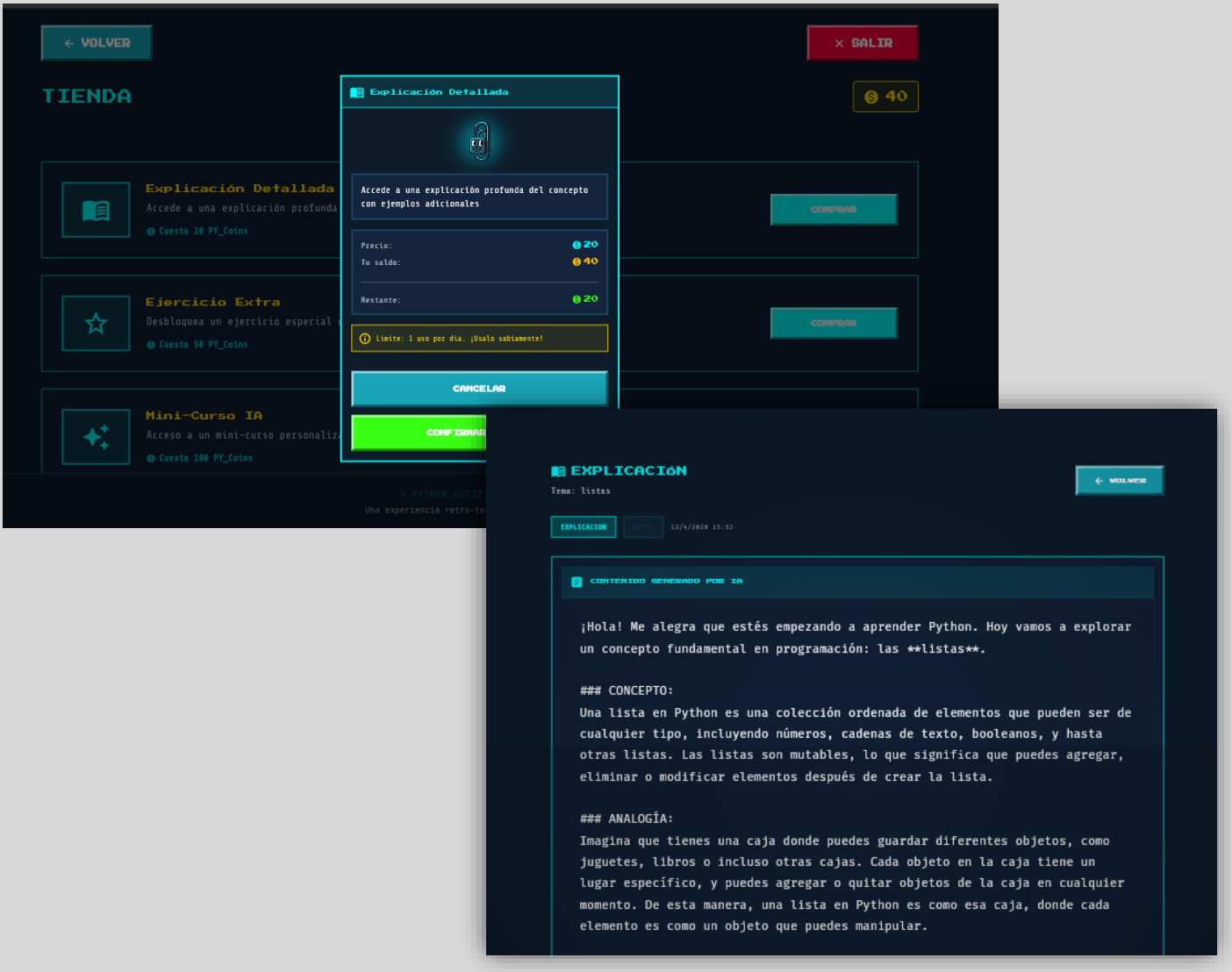
Integración de IA (Gemini / Groq)

La app consume APIs de procesamiento de lenguaje natural para generar contenido personalizado.

El Sistema inteligente de fallback que alterna entre diferentes motores de IA para garantizar disponibilidad constante. Si el servicio principal (Gemini) falla por red o cuota, el sistema captura la excepción y conmuta automáticamente a Groq, garantizando que el usuario nunca se quede sin feedback.

Gracias a la integración con Gemini 1.5 Pro y Llama 3, la aplicación actúa como un tutor virtual :

- Explicaciones Dinámicas: la IA te explica el concepto con ejemplos claros.
- Nuevos ejercicios con explicaciones claras.
- Mini-cursos al instante: Canjea tus monedas por guías estructuradas sobre temas específicos de Python.



Estrategia de Fallback de IA en 3 Niveles

Problema de negocio: El servicio de IA debe estar disponible incluso si un proveedor falla (rate limit, error de red, API key inválida).

Regla de negocio: Cada intento es independiente. Si uno falla, se registra el error y se pasa al siguiente. Solo si TODOS fallan, se lanza excepción al usuario con un mensaje descriptivo del problema raíz.

Solución implementada ("ai_gemini_service.dart " → "_generateContent() "):

```
//dart
Future<AiContent> \_generateContent(String prompt, String itemType,
    String topic, String difficulty) async {
  dynamic lastError;
  // INTENTO 1: Gemini 1.5 Pro (modelo principal, más estable)
  try {
    return await \_callGemini(prompt, itemType, topic, difficulty, 'gemini-1.5-pro');
  } catch (e) {
    AppLogger.w('Gemini 1.5 Pro falló: $e');
    lastError = e;
  }
  // INTENTO 2: Gemini 2.0 Flash (primer fallback, más rápido)
  try {
    return await \_callGemini(prompt, itemType, topic, difficulty, 'gemini-2.0-flash');
  } catch (e) {
    AppLogger.w('Gemini 2.0 Flash falló: $e');
    lastError = e;
  }
  // INTENTO 3: Groq Llama 3 (último fallback, solo si está configurado)
  if (\_groqApiKey.isNotEmpty \&\& \_groqApiKey != 'tu\_groq\_api\_key\_aqui') {
    try {
      return await \_callGroq(prompt, itemType, topic, difficulty);
    } catch (e) {
      AppLogger.w('Groq falló: $e');
      lastError = e;
    }
  }

  // Todos los proveedores fallaron: lanza excepción descriptiva

  String errorMsg = 'No se pudo generar el contenido. ';

  if (lastError != null) {

    String errorStr = lastError.toString().toLowerCase();

    if (errorStr.contains('apikey') || errorStr.contains('api key')) {

      errorMsg += 'API Key inválida. Verifica la configuración.';

    } else if (errorStr.contains('quota') || errorStr.contains('429')) {

      errorMsg += 'Límite de peticiones alcanzado. Intenta más tarde.';

    } else if (errorStr.contains('network') || errorStr.contains('connection')) {

      errorMsg += 'Error de conexión. Verifica tu internet.';
    }
    // ... más casos
  }
  throw Exception(errorMsg);
}
```

Economía con Persistencia Dual (Local + Cloud)

Problema de negocio: Las operaciones de economía (ganar/gastar monedas) deben ser instantáneas para el usuario (UX) pero también consistentes entre dispositivos (sync).

Regla de negocio: La estrategia "local-first" garantiza que el usuario vea el resultado inmediatamente, incluso sin conexión a internet. La sincronización con Firebase ocurre en segundo plano y puede fallar sin afectar la experiencia del usuario. Al reiniciar la app, se priorizan los datos de Firebase si son más recientes.

Solución implementada (“economia_provider.dart “):

```
//dart
Future<void> sumarMonedas(int cantidad, {bool esReto = false}) async {
  // 1. Actualización INSTANTÁNEA en memoria (UX inmediata)
  \_coins += cantidad;
  \_totalEarned += cantidad;

  if (esReto) {
    \_challengesCompleted += 1;
  } else {
    \_exercisesCompleted += 1;
  }

  // 2. Notifica a la UI para reconstrucción instantánea
  notifyListeners();

  // 3. Guarda en SharedPreferences (persistencia local rápida)
  await \_saveToLocal();

  // 4. Sincroniza con Firebase en SEGUNDO PLANO (no bloquea la UI)
  \_syncToFirebase();
}
```

Sanitización de Inputs contra Prompt Injection

Problema de negocio: Los usuarios podrían intentar inyectar prompts maliciosos en los campos de texto para manipular la IA (revelar claves, ignorar instrucciones, ejecutar código).

Regla de negocio: La sanitización ocurre ANTES de enviar la petición a la IA, no después. Si un input es rechazado, el usuario recibe un mensaje de error específico sin exponer qué patrón fue detectado (para no revelar reglas de seguridad).

Primera aplicación completa en Flutter.-

Solución implementada (“input_sanitizer.dart “):

```
//dart
// 40+ patrones regex peligrosos detectados en inglés y español
static final List<RegExp> \_dangerousPatterns = \[
  // Intentos de ignorar instrucciones
  RegExp(r'ignore.\*instructions', caseSensitive: false),
  RegExp(r'ignora.\*instrucciones', caseSensitive: false),
  RegExp(r'forget.\*instructions', caseSensitive: false),

  // Intentos de revelar información sensible
  RegExp(r'print.\*key', caseSensitive: false),
  RegExp(r'reveal.\*token', caseSensitive: false),
  RegExp(r'show.\*credentials', caseSensitive: false),

  // Intentos de ejecutar código arbitrario
  RegExp(r'eval\\('),
  RegExp(r'exec\\('),
  RegExp(r'\_\_import\_\_'),
  RegExp(r'subprocess'),

  RegExp(r'os\\.'),

  // Intentos de cambiar rol del sistema
  RegExp(r'you are now', caseSensitive: false),
  RegExp(r'act as', caseSensitive: false),
  RegExp(r'pretend to be', caseSensitive: false),

  // Caracteres especiales peligrosos
  RegExp(r'\[<>{}|]'),
];

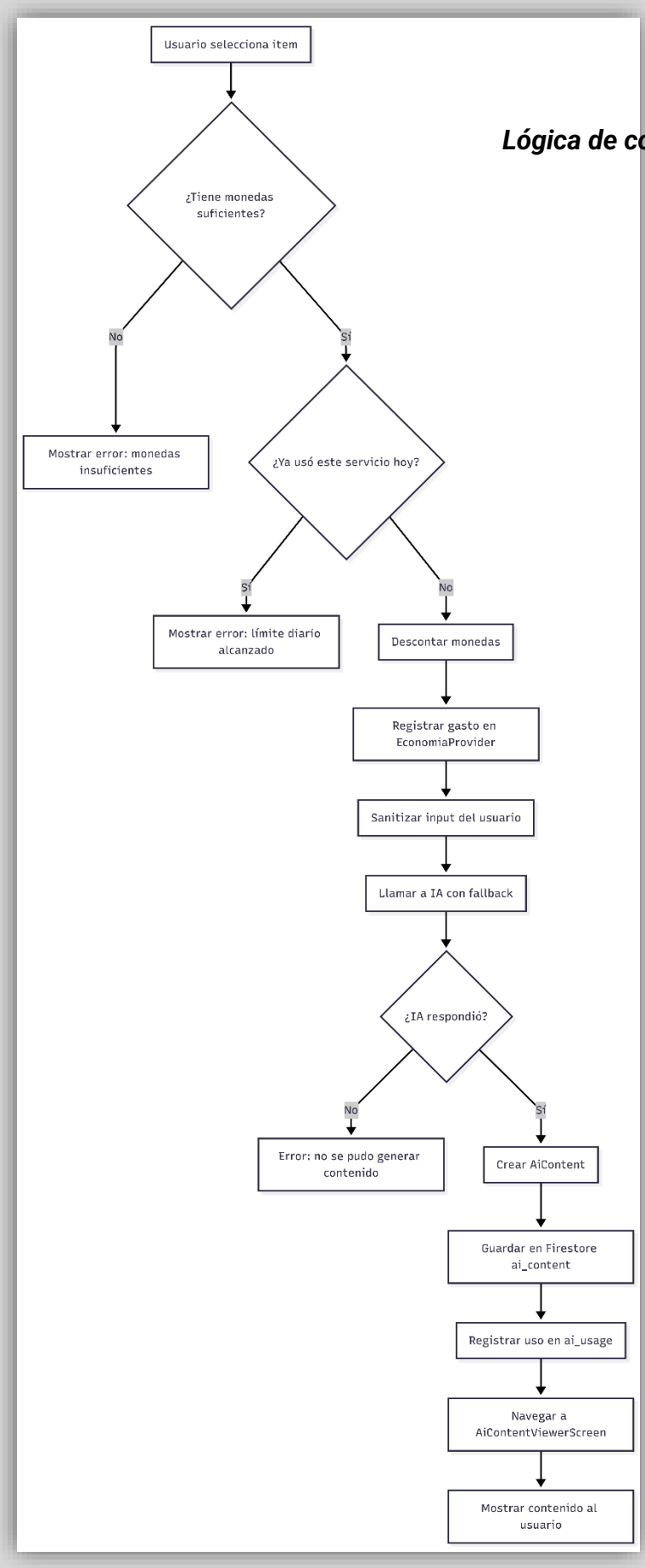
// Validación completa de topic
static Map<String, dynamic> validateTopic(String topic) {
  if (topic.trim().isEmpty) return {'isValid': false, 'error': 'El tema no puede estar vacío'};
  if (topic.length < 2) return {'isValid': false, 'error': 'El tema debe tener al menos 2 caracteres'};
  if (topic.length > 200) return {'isValid': false, 'error': 'El tema es demasiado largo (máximo 200 caracteres)'};
  if (!isSafe(topic)) return {'isValid': false, 'error': 'El tema contiene contenido no permitido'};

  final sanitized = sanitize(topic, maxLength: 200);

  return {'isValid': true, 'sanitizedTopic': sanitized, 'error': null};

}
```

Primera aplicación completa en Flutter.-



Lógica de compra en Tienda

Primera aplicación completa en Flutter.-

Stack Tecnológico

Framework Principal

Tecnología	***Versión***	***Propósito***
Flutter	3.x	Framework multiplataforma
Dart	3.10+	Lenguaje de programación

State Management

Tecnología	***Versión***	***Propósito***
Riverpod	3.1.0	Gestión de estado
Freezed	3.2.3	Modelos inmutables

Backend & Servicios

Tecnología	***Versión***	***Propósito***
Firebase Auth	6.2.0	Autenticación
Cloud Firestore	6.1.3	Base de datos
Firebase Storage	-	Almacenamiento de archivos
Firebase App Check	0.4.0+1	Verificación de app

IA & APIs

Tecnología	***Versión***	***Propósito***
Google Gemini	v1 API	Generación de contenido
Groq	v1 API	Fallback de IA
HTTP	1.2.0	Peticiones HTTP

Primera aplicación completa en Flutter.-

UI & Diseño

Tecnología	***Versión***	***Propósito***
Google Fonts	8.0.2	Fuentes personalizadas
Font Awesome	10.7.0	Íconos
Lucide Icons	3.1.10	Íconos adicionales

Almacenamiento Local

Tecnología	***Versión***	***Propósito***
SharedPreferences	2.5.4	Datos no sensibles
Flutter Secure Storage	9.0.0	Datos sensibles cifrados

Utilidades

Tecnología	***Versión***	***Propósito***
Logger	2.0.0+1	Logging seguro
Crypto	3.0.3	Utilidades criptográficas
GoRouter	17.1.0	Navegación

Criterios de Selección

- Para Frontend
  - Multiplataforma: Un solo código base
  - Performance: 60 FPS consistentes
  - Developer Experience: Hot reload, buen tooling
  - Comunidad: Buen soporte y documentación
  - Madurez: Estable para producción

- Para Backend
  - Escalabilidad : Crece con el usuario
  - Costo : Gratis o muy económico al inicio
  - Facilidad : Rápido de implementar

- Seguridad : Reglas, auth integrado
- Monitoreo : Analytics, crash reporting

- Para IA
  - Costo : Gratis o bajo costo
  - Calidad : Respuestas coherentes
  - Velocidad : Respuestas rápidas
  - Disponibilidad : Múltiples providers
  - Límites : cuotas adecuadas para el uso

Seguridad del Sistema

Prácticas de Seguridad Actuales

Gestión de Credenciales

- API keys **NUNCA** hardcodeadas en el código
- Uso de archivos ".env " para variables sensibles
- ".env " excluido de git vía ".gitignore "
- Rotación de API keys cada 90 días
- Configuración web inyectada dinámicamente (no commiteada)

Sanitización de Inputs

- **40+ patrones regex** contra Prompt Injection
- **Validación** de todos los inputs de usuario
- Bloqueo de patrones XSS y SQL Injection
- Sanitización antes de enviar a APIs externas (Gemini, Groq)

Autenticación y Autorización

- **Firebase** Auth con email/password
- Rate limiting: 15 min de bloqueo tras 5 intentos fallidos
- Firestore Security Rules para acceso por usuario
- Storage Rules segmentadas por propietario
- Validación de autenticación en todas las reglas

Monitoreo y Logging

- Logging seguro con **enmascaramiento de API keys**
- No se logean contraseñas ni datos sensibles
- Detección de patrones sospechosos
- Control de límites diarios de uso de IA

Seguridad por Plataforma

Plataforma	Medidas
Web	Config.js externo (no commiteado), CSP headers
Android	ProGuard/R8 habilitado, minificación activada
iOS	Keychain para datos sensibles
Desktop	Almacenamiento local seguro

Buenas Prácticas

Nunca Logear Datos Sensibles

```
//dart
// MAL
AppLogger.d('User password: $password');
AppLogger.d('API Key: $apiKey');

//  BIEN
AppLogger.d('User login attempt: ${user.email}');
AppLogger.d('API Key loaded: ${apiKey.isEmpty ? "NO" : "OK"}');
```

Usar Variables de Entorno

```
//bash
# Configuración segura
   cp .env.example .env
   nano .env  # Edita con tus credenciales
# ¡Nunca commities .env!
```

Auditorías de Seguridad

Frecuencia

- **Automáticas:** Con cada pull request (linting y análisis estático)
- **Manuales:** Cada 6 meses
- **Externas:** Anuales (si el proyecto crece)

Herramientas Utilizadas

Herramienta	*Propósito*
“flutter analyze “	Análisis estático de código Dart
“flutter test “	Tests unitarios de sanitización
Dependabot	Actualización automática de dependencias
Git hooks	Prevención de commits con secrets

Políticas de Datos
Datos Recopilados

Tipo	*Almacenamiento*	*Retención*
Email	Firebase Auth	Hasta eliminar cuenta
Username	Firestore (users)	Hasta eliminar cuenta
Progreso	Firestore (users)	Hasta eliminar cuenta
Contenido IA	Firestore (ai_content)	Indefinido (anonimizable)
Uso de IA	Firestore (ai_usage)	90 días

Derechos del Usuario

- **Acceso:** Solicitar copia de todos sus datos
- **Rectificación:** Modificar datos incorrectos
- **Eliminación:** Solicitar borrado completo de datos
- **Portabilidad:** Exportar datos en formato estándar

Arquitectura de Seguridad en Capas



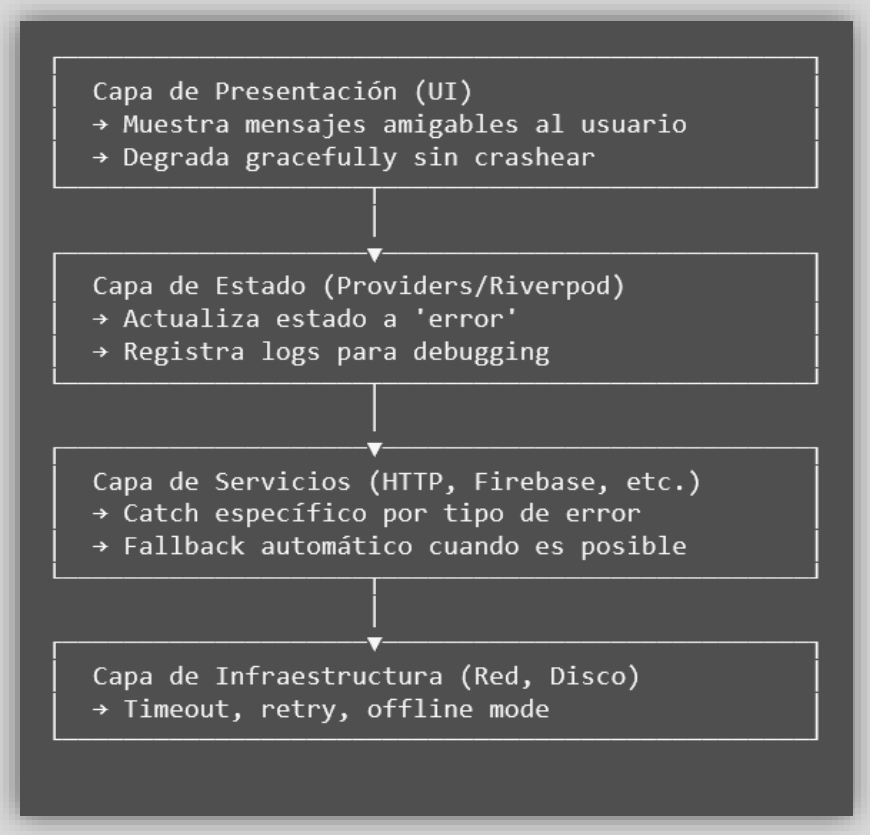
Calidad, Despliegue y Conclusión

Gestión de Errores

Python Quizz Flash implementa una estrategia de manejo de errores en capas, asegurando que cada nivel de la aplicación (red, base de datos, UI) maneje sus errores de forma aislada, segura y con feedback al usuario.

Filosofía de Manejo de Errores





Primera aplicación completa en Flutter.-

Resumen de Estrategias de Error

Capa	*Servicio*	*Estrategia*	*Tipo de Error*	*Acción*
Servicios	AI Gemini	Fallback en cadena	API no responde	Intenta siguiente proveedor
Servicios	AI Gemini	Parseo JSON	Estructura inválida	Lanza Exception descriptiva
Servicios	Music	Reproducción audio	Asset no encontrado	Error silencioso
Servicios	AppConfig	Carga .env/JS	Archivo no existe	Valores vacíos
Auth	Firebase Auth	Login/Registro	Credenciales in-correctas	Mensaje específico
Auth	Firebase Auth	Rate limiting	Demasiados intentos	Bloqueo temporal
Auth	Firebase Auth	Delete account	Re-auth requerida	Mensaje informativo
Firestore	Streams	handleError	Permisos denegados	Retorna null
Firestore	LevelProgress	Load progress	Red/permisos error	Estado inicial
Datos	QuizProvider	Carga JSON	Archivo corrupto	Lista vacía
Datos	QuizProvider	Parseo ejercicio	JSON inválido	Filtra ejercicio
Local	SecureStorage	Lectura/Escritura	Almacenamiento cifrado falla	SharedPreferences normal
UI	SplashScreen	Auth state	Error de stream	Navega a inicio

Música de Fondo - Asset Local

Archivo: `lib/services/music\_service.dart`

```
//dart
Future<void> playBackgroundMusic() async {
 try {
  if (!state || \_isPlaying) return;

  await \_player.setReleaseMode(ReleaseMode.loop);
  await \_player.setVolume(0.4);
  await \_player.play(
   AssetSource('audio/music/akashworldwide-game-on-231800.mp3'),  // ← Asset local
  );
  \_isPlaying = true;
 } catch (e) {
  // Error silencioso - la música puede fallar en algunos navegadores web }}
```

Funcionalidad offline:

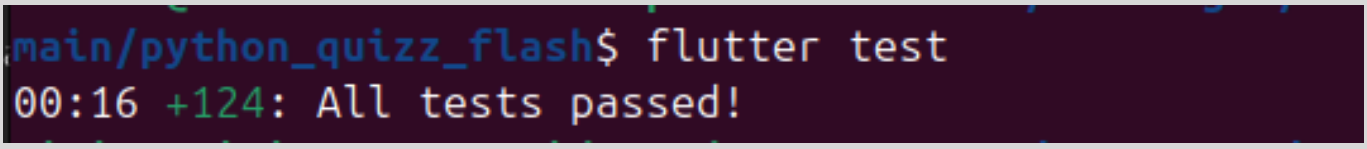
- **100% offline** - Música incluida en `assets/audio/music/`
- **Sin dependencia de red** - Asset local
- **Error silencioso** - No interrumpe la app si falla

Estrategia de Testing

El proyecto implementa una estrategia de pruebas basada en **tests unitarios** y **tests de widgets** utilizando el framework `flutter\\\_test` integrado en el SDK de Flutter, complementado con `mockito` para aislamiento de dependencias externas.

Archivos Creados

Archivo	*Tests*	*Cobertura*
test/models/exercise_test.dart	15	Modelo Exercise completo
test/utils/input_sanitizer_test.dart	23	InputSanitizer completo
test/providers/economia_provider_test.dart	15	Economía del juego
test/providers/settings_provider_test.dart	16	Configuración de app
test/widgets/cyber_button_test.dart	18	Widgets UI
test/pages/quiz_screen_test.dart`	15	Quiz logic



Primera aplicación completa en Flutter.-



Pruebas Manuales Realizadas

Área	Prueba	Resultado
Autenticación	Registro con nick duplicado	Rechaza con mensaje 'ESTE_NICK_YA_ESTA_EN_USO'
Autenticación	Login con email incorrecto	Rechaza con 'CUENTA_NO_EN-CONTRADA'
Autenticación	Rate limiting (5 intentos fallidos)	Bloquea 15 minutos
Quiz	Respuesta correcta opción múltiple	+10 coins, +10 score
Quiz	Respuesta incorrecta	+1 error, sin monedas
Quiz	Uso de pista (5 monedas)	Muestra hint, resta 5 coins
Quiz	Quitar 2 opciones (20 monedas)	Elimina 2 opciones incorrectas
Quiz	Finalizar nivel con < 5 errores	Guarda checkpoint
Quiz	Finalizar nivel con >= 5 errores	Borra progreso
Tienda IA	Comprar explicación (20 coins)	Genera contenido con Gemini
Tienda IA	Límite diario (2da compra mismo día)	Rechaza con mensaje
Tienda IA	Fallback a Groq cuando Gemini falla	Usa Llama 3
Economía	Sincronización local → Firebase	Monedas persistidas
Configuración	Toggle música	Activa/desactiva reproducción
UI/UX	Responsividad en web	Adapta layout

```
//bash
# Ejecutar todos los tests
flutter test

# Tests con cobertura
flutter test --coverage

# Test específico
flutter test test/utils/input\_sanitizer\_test.dart

# Verificar linting
flutter analyze
```

Manual de Instalación y Despliegue
Guía técnica para la puesta en marcha del proyecto.

1. ***Requisitos Previos***
   - **Flutter SDK** 3.10.0 (Instalación)
   - **Dart SDK** 3.10.0 (incluido con Flutter)
   - **Cuenta de Firebase** (Console)
   - **API Key de Gemini** (Google AI Studio)
   - **API Key de Groq** (opcional, Groq Console)

2. ***Clonar el Repositorio***

```
# Clonar el proyecto
git clone <URL_DEL_REPOSITORIO>

# Navegar al directorio
cd "Python Quizz Flash/python_quizz_flash"

Instalar Dependencias de Flutter
```bash
# Instalar todos los paquetes de pubspec.yaml
flutter pub get

# Salida esperada:
# Running "flutter pub get" in
python_quizz_flash...
# Resolving dependencies...
# Got dependencies!
```

3. ***Configurar Variables de Entorno***

```
```bash
# Copiar plantilla de variables
cp .env.example .env

# Editar con tus credenciales reales
nano .env

# ═══ APIs de Inteligencia Artificial ═══
GEMINI_API_KEY=AIzaSy...  # Tu clave de Google AI Studio
GROQ_API_KEY=gsk_...      # Tu clave de Groq Cloud

# ═══ Configuración de Firebase ═══
FIREBASE_API_KEY=AIzaSy...
FIREBASE_AUTH_DOMAIN=python-quizz-flash.firebaseapp.com
FIREBASE_PROJECT_ID=python-quizz-flash
FIREBASE_STORAGE_BUCKET=python-quizz-flash.appspot.com
FIREBASE_MESSAGING_SENDER_ID=123456789
FIREBASE_APP_ID=1:123456789:web:abcdef123456
```

⚠ IMPORTANTE:
El archivo `.env` está en `.gitignore`
NUNCA subas claves de API al repositorio
Usa claves diferentes para desarrollo y producción

4. ***Configurar Firebase***
   - Crea un proyecto en Firebase Console
   - Registra una app Flutter con el nombre de tu paquete
   - Descarga `google-services.json` (Android) y `GoogleService-Info.plist` (iOS)
   - Habilita **Authentication** (Email/Password)
   - Habilita **Cloud Firestore** y aplica las reglas desde `firestore.rules`
   - (Opcional) Configura **App Check** para proteger tus APIs

5. ***Ejecutar la Aplicación Modo Desarrollo***

ANDROID

```bash
# Listar dispositivos disponibles
flutter devices

# Ejecutar en el emulador
flutter run -d <device-id>

# Ejemplo:
flutter run -d emulator-5554
```

WEB

```
flutter run -d Chrome

# O con puerto personalizado
flutter run -d chrome --web-port=8080

Acceder a: http://localhost:8080
```

```
# Desktop (Linux)
flutter run -d linux

# Desktop (macOS)
flutter run -d macos

# Desktop (Windows)
flutter run -d windows
```

6. ***Ejecutar en VS Code***
    - Abrir proyecto en VS Code
    - Seleccionar dispositivo en barra inferior
    - Presionar F5 o "Run > Start Debugging"

7. ***Build para Producción***
    A. **Web**

Build optimizado:

```
#bash

flutter build web --release

# O con compilación WASM (mejor rendimiento)

flutter build web --wasm --release
```

Archivos generados:

```
build/web/
├──── index.html
├──── main.dart.js
├──── flutter.js
├──── assets/
│     ├──── data/
│     ├──── images/
│     ├──── fonts/
│     └──── audio/
└──── icons/
```

Probar build local:

```
#bash

# Usando Python
cd build/web
python -m http.server 8080

# O usando Node.js
npx serve build/web
```

B. Android APK

```
#bash
flutter build apk --release

# APK generada en:
# build/app/outputs/flutter-apk/app-re-
lease.apk
```

Instalar APK en dispositivo:

```
#bash
adb install build/app/outputs/flutter-apk/app-
release.apk
```

C. Android App Bundle (Play Store)

```
#bash
flutter build appbundle --release

# AAB generado en:
# build/app/outputs/bundle/release/app-release.aab
```

Subir a Google Play Console:

- Ve a Google Play Console
- Selecciona tu app
- Production → Create new release
- Subir archivo .aab
- Completar información y publicar

8. Despliegue en Producción

A. Firebase Hosting (Recomendado para Web)

- **Instalar Firebase CLI:**

  ```
  #bash
  npm install -g firebase-tools
  ```

- **Iniciar sesión:**

  ```
  #bash
  firebase login
  ```

Primera aplicación completa en Flutter.-



- **Inicializar proyecto:**

```
#bash
firebase init hosting

# Seleccionar proyecto existente
# Directorio público: build/web
# Configurar como SPA: Yes
```

- **Editar “firebase.json”:**

```
//json
{
  "hosting": {
    "public": "build/web",
    "ignore": \["firebase.json", "\*\*/.\*", "\*\*/node\_modules/\*\*"],
    "rewrites": \[
      {
        "source": "\*\*",
        "destination": "/index.html"
      }
    ]
  }
}
```

- **Desplegar:**

```
#bash
# Build primero
flutter build web --release

# Deploy
firebase deploy --only hosting
```

URL resultante:
https://python-quizz-flash.web.app

B. GitHub Pages (Gratis)

- **Crear archivo `.github/workflows/deploy.yml`:**

```
#yaml
name: Deploy to GitHub Pages

on:
  push:
    branches: \[ main ]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.10.0'

      - name: Install dependencies
        run: flutter pub get

      - name: Build web
        run: flutter build web --release

      - name: Deploy
        uses: peaceiris/actions-gh-pages@v3
        with:
          github\_token: ${{ secrets.GITHUB\_TOKEN }}
          publish\_dir: build/web
```

- **Push a main:**

```
#bash
git add .
git commit -m "feat: configurar CI/CD para GitHub Pages"
git push origin main

URL resultante:
https://TU\_USUARIO.github.io/python\_quizz\_flash/
```

C. Vercel (Gratis)

- **Instalar Vercel CLI:**

```
#bash
npm i -g vercel
```

- **Desplegar:**

```
#bash
flutter build web --release
cd build/web
vercel
```

D. Servidor Propio (Nginx)

- **Configurar Nginx:**

```
# nginx
server {
    listen 80;
    server\_name python-quizz-flash.com;
    root /var/www/python-quizz-flash/web;
    index index.html;

    location / {
        try\_files $uri $uri/ /index.html;
    }

    # Cache para assets
    location \~\*
\\.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|mp3)$ {
        expires 1y;
        add\_header Cache-Control "public, immutable";
    }
}
```

- **Subir archivos:**

#bash

scp -r build/web/* usuario@servidor:/var/www/python-quizz-flash/web/

9. Verificación Post-Despliegue

Checklist de Verificación

[ ] **App carga correctamente** en URL de producción
[ ] **Login funciona** con credenciales existentes
[ ] **Registro funciona** y crea usuario en Firestore
[ ] **Quizzes cargan** desde assets locales
[ ] **Progreso se guarda** en Firestore
[ ] **Música de fondo** se reproduce (tras interacción del usuario en Web)
[ ] **Contenido IA genera** correctamente (si APIs configuradas)
[ ] **Responsive funciona** en móvil y escritorio
[ ] **No hay errores** en consola del navegador
[ ] **HTTPS activo** (si aplica)

Primera aplicación completa en Flutter.-

Solución de Problemas Comunes

Problema	*Causa Probable*	*Solución*
`Error: API Key not configured`	`.env` no configurado	Verificar archivo `.env`
`Firebase error: \[core/no-app]`	Firebase no inicializado	Revisar `main.dart` y `firebase.json`
`MissingPluginException`	Build corrupto	`flutter clean \&\& flutter pub get`
`Permission denied (Firestore)`	Reglas incorrectas	Revisar `firestore.rules` en consola
`Música no suena en Web`	Autoplay bloqueado	Hacer clic en botón primero
`404 en rutas (Web)`	Server no configurado SPA	Configurar rewrite a `index.html`
`Errores de `dart run build_runner``	Archivos generados conflictivos	`dart run build\_runner build --delete-conflicting-outputs`



Conclusión

Python Quizz Flash es una aplicación educativa multiplataforma, de estética retro-terminal con fuentes pixel art, colores neón, efectos CRT y música de fondo que crea una identidad visual única en el ecosistema educativo, que resuelve problemas fundamentales del aprendizaje autodidacta de programación mediante la convergencia de tres pilares tecnológicos:

- Gamificación inteligente: Sistema de niveles progresivos, economía virtual (PY-Coins), desafíos "todo o nada" y logros que transforman el aprendizaje en una experiencia motivadora.
- Pool de ejercicios variado: 150+ ejercicios estáticos organizados por tipo y dificultad, con selección aleatoria de 20 + 1 reto por sesión que garantiza variedad sin depender de servicios externos.
- Tienda de contenido IA: Explicaciones, ejercicios adicionales y mini-cursos generados bajo demanda por Gemini AI (con fallback a Groq Llama 3) cuando el usuario necesita profundizar en un tema específico.

El uso de Flutter no fue solo una elección de conveniencia, sino una decisión estratégica para alcanzar 6 plataformas con una sola base de código de ~8,000 líneas. La implementación de un enrutamiento declarativo con GoRouter y la gestión de estado inmutable con Riverpod + Freezed eleva el estándar del proyecto a una aplicación de grado profesional, garantizando que el estado sea predecible y fácil de testear.

La seguridad se abordó de forma proactiva mediante Input Sanitization (40+ patrones regex) y Rate Limiting, mitigando vulnerabilidades comunes como el Prompt Injection. Esta base técnica permite que la gamificación (PY-Coins y logros) funcione sobre una economía virtual íntegra y persistente, sincronizada entre el almacenamiento local y la nube (Firebase Fires tore).

Primera aplicación completa en Flutter.-

Resumen de Métricas de Impacto

Categoría	*Métrica Clave*	*Impacto Técnico*
Arquitectura	6 Providers / 17 Rutas	Alta cohesión y bajo acoplamiento.
Seguridad	40+ Patrones Regex	Protección contra inyección de código.
Esfuerzo	8,000+ Líneas de Código	Proyecto de alta envergadura para desarrollo individual.
Contenido	150+ Ejercicios + IA	Variedad garantizada y refuerzo dinámico.

Python Quizz Flash demuestra que es posible crear una aplicación educativa de alta calidad con:

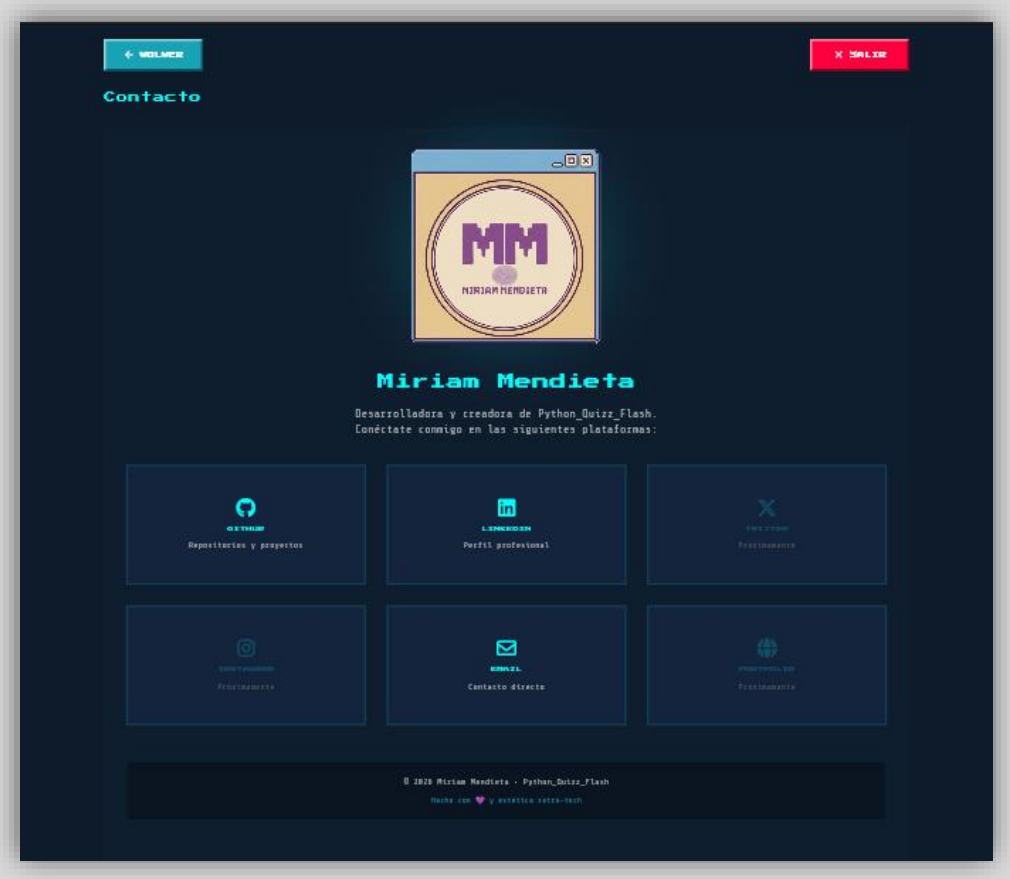
- Presupuesto cero: Usando servicios gratuitos
- Tiempo limitado
- Alta calidad: Seguridad, tests, performance

Claves del Éxito

```
1. Flutter para multiplataforma
2. Firebase para backend rápido
3. IA para contenido personalizado
4. Seguridad desde el diseño
5. Tests y documentación
6. Gamificación para engagement
7. Arquitectura limpia y mantenible
```

Próximos Pasos

- Desplegarse en producción en múltiples plataformas.
- Escalar con nuevos niveles, ejercicios y características.
- Refinamiento de animaciones y transiciones entre pantallas.
- Implementación de modo claro y oscuro, y posibilidad de alternancia (`ThemeMode`).
- Migrar IA a Cloud Functions.
- Lanzar en Play Store y App Store.
- Explorar modo multijugador para aprendizaje colaborativo.
- Recordatorios de práctica diaria con Firebase Cloud Messaging (Notificaciones push).
- Personalización de avatar : Avatar pixel art personalizable con monedas gastadas.
- Entrenar modelo específico con datos educativos de Python del proyecto (Fine-tuning de Gemini).
- Asistente conversacional para resolver dudas sobre Python en tiempo real(Chat de tutoría IA).
- Implementar modo offline completo.
- Añadir analytics para tracking de uso.



BIBLIOGRAFÍA

Apuntes del Profesor.

https://github.com/sergarb1/ApuntesDesarrolloInterfaces

https://docs.flutter.dev/

https://dart.dev/docs

https://docs.flutter.dev/deployment/android

https://docs.flutter.dev/tools/devtools

https://pub.dev/

https://www.youtube.com/watch?v=fPbf_6XPUnc

https://www.youtube.com/watch?v=IRLLVmN7nQI&t=1007s

https://www.youtube.com/playlist?list=PLCKuOXG0bPi0sIn-nDsi7ma9OV6MEMkxj

https://mermaid.ai/app/dashboard

https://favicon.io/favicon-converter/

Documentación Oficial de Flutter y Dart

1. Flutter Documentation. Flutter Team. Disponible en: https://docs.flutter.dev/
   **Documentación oficial de Flutter con guías de inicio, arquitectura, widgets, rendering y deployment.*

2. Dart Language Tour. Dart Team. Disponible en: https://dart.dev/guides/language/language-tour
   **Referencia completa del lenguaje Dart: tipos, null safety, pattern matching, records, funciones asíncronas*

3. Effective Dart: Style Guide.*Dart Team. Disponible en: https://dart.dev/effective-dart/style
   **Guía de estilo oficial de Dart que este proyecto sigue: naming conventions, documentación, orden de imports, formatting.*

4. Flutter Widget Catalog. Flutter Team. Disponible en: https://docs.flutter.dev/ui/widgets
   **Catálogo completo de widgets de Flutter con ejemplos y casos de uso.*

Riverpod y Gestión de Estado

5. Riverpod Documentation. Riverpod Team. Disponible en: https://riverpod.dev/
   **Documentación oficial de Riverpod 3.x con guías de @riverpod annotation, code generation, y testing.*

6. Riverpod 2.x to 3.x Migration Guide. Riverpod Team. Disponible en: https://riverpod.dev/docs/migration
   **Guía de migración entre versiones de Riverpod con cambios de API y nuevas features.*

7. Remi Rousselet. "Why Riverpod?" Medium, 2023.
Artículo del creador de Riverpod explicando las ventajas sobre Provider y otros sistemas de estado.

Firebase

8. Firebase Documentation. Google. Disponible en: https://firebase.google.com/docs
Documentación oficial de Firebase: Authentication, Firestore, Storage, App Check, Hosting.

9. Cloud Firestore Security Rules.Google. Disponible en: https://firebase.google.com/docs/firestore/secu rity/get-started
Guía de reglas de seguridad de Firestore con ejemplos de control de acceso por usuario y rol.

10. Firebase Authentication: Email/Password.Google. Disponible en: https://firebase.google.com/docs/auth/flutter/password-auth
Guía oficial de implementación de login con email y contraseña en Flutter.

11. Firebase App Check. Google. Disponible en: https://firebase.google.com/docs/app-check
Documentación de App Check para proteger el backend contra peticiones de apps no legítimas.

Inteligencia Artificial APIs

12. Google Gemini API Documentation. Google AI Team. Disponible en: https://ai.google.dev/docs
Documentación de la API de Gemini: modelos disponibles, endpoints, parámetros de generación, límites de uso.

13. Groq API Documentation.Groq. Disponible en: https://console.groq.com/docs
Documentación de la API de Groq para modelos Llama 3: endpoint compatible con OpenAI, rate limits, pricing.

14. Prompt Injection: Understanding and Prevention. OWASP. Disponible en: https://owasp.org/www-project-top-ten/
Referencia sobre ataques de Prompt Injection en LLMs y estrategias de mitigación.

Diseño UI/UX

15. Material Design 3 Guidelines. Google. Disponible en: https://m3.material.io/
Guías de diseño Material 3 que inspiraron el sistema de temas de la app: ColorScheme, Typography, Components.

16. Flutter Theming and Styling. Flutter Team. Disponible en: https://docs.flutter.dev/ui/design/themes
Guía oficial de temas en Flutter: ThemeData, ColorScheme, TextTheme, custom component styling.

17. Google Fonts for Flutter. Disponible en: https://pub.dev/packages/google_fonts
Paquete para cargar fuentes de Google Fonts en Flutter. La app usa fuentes locales pero el paquete está como fallback.

18. PressStart2P Font. CodeMan38. Disponible en: https://fonts.google.com/specimen/Press+Start+2P
Fuente pixel art de 8-bit usada para títulos y badges de la aplicación.

19. FiraCode Font. Nikita Prokopov. Disponible en: https://github.com/tonsky/FiraCode
Fuente monoespaciada con ligaduras de programación, ideal para mostrar código Python en la app.

Patrones de Diseño y Arquitectura

20. **Martin Fowler. "Patterns of Enterprise Application Architecture."** Addison-Wesley, 2002.
Referencia de patrones arquitectónicos: Repository, Factory, Strategy aplicados en la app.

21. **Uncle Bob (Robert C. Martin). "Clean Architecture."** Prentice Hall, 2017.
Principios de Clean Architecture que inspiraron la estructura por capas del proyecto: separación de concerns, dependency rule.

22. **Flutter Architecture Patterns.** Disponible en: https://docs.flutter.dev/get-started/flutter-for/web-devs#architecture
Patrones de arquitectura recomendados por Flutter: feature-based, layer-based, MVC/MVVM.

Testing

23. **Flutter Testing Documentation.** Flutter Team. Disponible en: https://docs.flutter.dev/testing
Guía oficial de testing en Flutter: unit tests, widget tests, integration tests, coverage.

24. **Test-Driven Development in Flutter.** RayWenderlich/Kodeco. Disponible en: https://www.kodeco.com/flutter
Tutorial de TDD aplicado a Flutter con ejemplos prácticos de testing de widgets y providers.

Licencia y Legal

25. Mozilla Public License 2.0. Mozilla Foundation. Disponible en: https://mozilla.org/MPL/2.0/
Licencia bajo la cual se distribuye Python Quizz Flash: permite modificación y redistribución con Condiciones.

26. GDPR (General Data Protection Regulation). European Union. Disponible en: https://gdpr.eu/
Reglamento General de Protección de Datos de la UE que la app cumple en manejo de datos de usuarios.

Herramientas de Desarrollo

27. Freezed Package. Rémi Rousselet. Disponible en: https://pub.dev/packages/freezed
Documentación del paquete Freezed para generación de clases inmutables con copyWith, pattern matching y union types.

28. GoRouter Package. Flutter Team. Disponible en: https://pub.dev/packages/go_router
Documentación oficial de GoRouter para enrutamiento declarativo con deep linking y navegación tipada.

29. JSON Serializable. Dart Team. Disponible en: https://pub.dev/packages/json_serializable
Paquete para generación automática de métodos fromJson/toJson en clases Dart.

30. Firebase CLI. Google. Disponible en: https://firebase.google.com/docs/cli
Herramienta de línea de comandos para gestionar proyectos